GRIMOIRE VOLUME III-C / FULL FORMULA EXPANSION

SOL FABLE–GENESIS

The Pure LaTeX Tower, v2.0

From finite generators and Corinth harmonics to the Light Matrix, Schur commutants, and the Step-4 representation fork

generator $\longrightarrow$ orbit $\longrightarrow$ invariant $\longrightarrow$ representation $\longrightarrow$ commutant $\longrightarrow$ Dirac constraints $\longrightarrow$ $\underbrace{\text{trivial}}_{\text{mathematical joke}}$

Vladyslav Savytskyy
with the SOL algebraic audit

Buenos Aires + Ancient Korinthos, 2026

Honest central statement. The topological, Eisenstein, Fibonacci, Light-Matrix, and conditional Schur-commutant rungs close. The supplied finite-bimodule specification yields an exact 272-dimensional base and exact 16/8 rank sandwiches in a source-aligned unit-completed model. The standard finite algebra remains representation-sensitive in the supplied reconstruction. The global Step-4 selection theorem is therefore marked *trivial* only in the traditional mathematician's sense: the proof has been left to the reader, because it does not yet exist here.

The equation may be exact; the naked numeral is incomplete. P=12. $\chi = 2$.

Contents

Abstract

This document expands the complete mathematical tower developed across the supplied files and conversation. It distinguishes exact identities from finite computations, conditional representation theorems, design analogies, and open physical claims. The exact core consists of: the fullerene curvature identity forcing twelve pentagons; the Eisenstein norm and Goldberg count formulas; the Fibonacci selector; the symmetric-square Light Matrix with spectrum $\{\phi^2, 1, -1, \phi^{-2}\}$; its Lorentzian invariant forms; and a planar-group-algebra no-go followed by a Schur-commutant construction of $\mathbb{C} \oplus \mathbb{H} \oplus M_3(\mathbb{C})$. The finite-bimodule core derives the 272-dimensional unconstrained real Dirac space and reconstructs exact 16- and 8-dimensional order-one spaces in the declared source-aligned and Pati–Salam models by finite-field rank sandwiches. A literal reading of the phrase "the anti-lepton colour is fixed by the identity" is affine, not linear; one linear unit completion reproduces the displayed source table, whereas one natural standard-A_F completion gives a different computed rank. The final selection problem is therefore written as trivial, explicitly as a joke, while its status remains open.

1 Status grammar: what each symbol is allowed to mean

The tower uses five logically distinct statuses:

$$\boxed{\text{EXACT}} \neq \boxed{\text{COMPUTED}} \neq \boxed{\text{CONDITIONAL}} \neq \boxed{\text{DESIGN}} \neq \boxed{\text{OPEN}}.$$

An exact equality is a statement internal to a declared formal model. A numerical certificate has the form

$$\widetilde{x} \doteq_{\mathcal{E},N,p,\varepsilon} x, \qquad \varepsilon = \frac{|\widetilde{x} - x|}{\max(1, |x|)},$$

where $\mathcal{E}$ records the execution environment, N the depth or budget, and p the arithmetic precision. A conditional theorem is exact only after its representation data have been supplied. A design relation is a renderer, analogy, or interface choice. An open statement may be written

$$\boxed{\text{trivial}}$$

only as the standard mathematical joke; the joke changes no status bit.

2 Rung I: generator, orbit, attractor, and finite render

2.1 The abstract iterated-function system

Let (X, d) be a complete metric space and let

$$f_j : X \to X, \qquad j = 1, \ldots, B,$$

be contractions with Lipschitz constants $0 < r_j < 1$. On the hyperspace $\mathcal{K}(X)$ of nonempty compact subsets, define the Hutchinson operator

$$\mathscr{H}(K) \coloneqq \bigcup_{j=1}^{B} f_j(K).$$

With the Hausdorff metric d_H,

$$d_H(\mathscr{H}(K_1), \mathscr{H}(K_2)) \le r_{\max} d_H(K_1, K_2), \qquad r_{\max} = \max_j r_j < 1.$$

Hence $\mathscr{H}$ has a unique compact fixed point

$$K_\star = \mathscr{H}(K_\star) = \bigcup_{j=1}^{B} f_j(K_\star).$$

Under the open-set condition, the similarity dimension D satisfies

$$\boxed{\sum_{j=1}^{B} r_j^D = 1.}$$

For equal ratios $r_j = \rho$,

$$\boxed{D = \frac{\log B}{\log(1/\rho)}.}$$

2.2 One similarity map and the logarithmic spiral

For

$$f(z) = c + \lambda(z - c), \qquad \lambda = \rho e^{i\Delta}, \qquad 0 < \rho < 1,$$

the discrete orbit is

$$z_n = c + \lambda^n (z_0 - c).$$

It is countable, so

$$\dim_H \{z_n : n \in \mathbb{N}\} = 0.$$

The continuous logarithmic spiral

$$\Gamma(t) = c + a e^{(b+i)t}$$

obeys

$$\Gamma(t + \tau) - c = e^{(b+i)\tau}\big(\Gamma(t) - c\big),$$

and in polar form

$$r(\theta) = r_0 e^{b\theta}.$$

Its tangent makes a constant angle α with the radial direction:

$$\tan\alpha = \frac{1}{|b|}, \qquad \cot\alpha = |b|.$$

As a smooth curve,

$$\boxed{\dim_H \Gamma = 1.}$$

Self-similarity does not imply a noninteger Hausdorff dimension.

2.3 Complex dimensions of the scale string

For scales

$$\ell_n = \ell_0 \rho^n,$$

define

$$\zeta_\rho(s) = \sum_{n=0}^{\infty} \ell_n^s = \frac{\ell_0^s}{1 - \rho^s}.$$

Its poles satisfy

$$\rho^s = 1,$$

therefore

$$\boxed{s_k = \frac{2\pi i k}{\log \rho}, \qquad k \in \mathbb{Z}.}$$

With multiplicity B^n,

$$\zeta_{B,\rho}(s) = \frac{\ell_0^s}{1 - B\rho^s},$$

so

$$\boxed{s_k = D + \frac{2\pi i k}{\log(1/\rho)}, \qquad D = \frac{\log B}{\log(1/\rho)}.}$$

The imaginary spacing produces log-periodicity in $\log \ell$.

2.4 The finite-render chain

A browser does not draw an infinite orbit. For an escape radius R and budget N it computes

$$\tau_{R,N}(z_0) = \min\Big(\{n \le N : |F^{\circ n}(z_0)| > R\} \cup \{N + 1\}\Big).$$

The displayed pixel is

$$\mathcal{P}_{N,p,\mathcal{E}}(z_0) = \mathcal{C}\big(\tau_{R,N}(z_0), F^{\circ N}(z_0); p, \mathcal{E}\big).$$

Thus

$$\boxed{F \neq \{F^{\circ n}\}_{n \ge 0} \neq \tau_{R,N} \neq \mathcal{P}_{N,p,\mathcal{E}}.}$$

3 Rung II: the Corinth image as a finite harmonic object

Let a rectified image be a function

$$I : \Omega \subset \mathbb{R}^2 \to \mathbb{R}^q,$$

with candidate centre $c = (c_x, c_y)$. Its polar pullback is

$$I_c(r, \theta) = I\big(c_x + r\cos\theta, c_y + r\sin\theta\big).$$

For a scalar feature $g(I)$, define the angular coefficient

$$\widehat{I}_m(r) = \frac{1}{2\pi}\int_0^{2\pi} g(I_c(r,\theta))\mathrm{e}^{-\mathrm{i}m\theta}\, d\theta,$$

and annular power

$$P_m[r_1, r_2] = \int_{r_1}^{r_2} |\widehat{I}_m(r)|^2\, w(r)\, dr.$$

If the image were exactly C_N-invariant,

$$I_c\left(r, \theta + \frac{2\pi}{N}\right) = I_c(r,\theta),$$

then

$$\widehat{I}_m(r) = \mathrm{e}^{-2\pi\mathrm{i}m/N}\widehat{I}_m(r),$$

which implies

$$\boxed{\widehat{I}_m(r) = 0 \quad \text{unless} \quad N \mid m.}$$

The supplied finite image audit finds dominant modes

$$\boxed{m_{\text{inner}} = 14}, \qquad \boxed{m_{\text{outer}} = 8}.$$

Across centre perturbations and nearby annuli, the inner edge target was strongest in 168/175 trials and the outer edge target in 175/175 trials. Therefore the correct statement is

$$\boxed{\text{robust finite angular harmonics} \quad \text{rather than} \quad \text{an exact archaeological symmetry theorem.}}$$

4 Rung III: Euler, fullerene closure, and discrete curvature

Let f_p denote the number of p-gonal faces of a connected trivalent graph cellularly embedded in S^2. Then

$$3V = 2E, \qquad \sum_{p\geq 3} p f_p = 2E, \qquad V - E + \sum_{p\geq 3} f_p = 2.$$

Eliminate V and E:

$$\begin{aligned} V - E + F &= 2, \\ \frac{2E}{3} - E + F &= 2, \\ -E + 3F &= 6, \\ 2E &= 6F - 12, \\ \sum_p p f_p &= 6\sum_p f_p - 12. \end{aligned}$$

Hence

$$\boxed{\sum_{p\geq 3}(6-p) f_p = 12.}$$

For pentagons and hexagons only,

$$(6-5)P + (6-6)H = 12,$$

so

$$\boxed{P = 12.}$$

Then

$$5P + 6H = 2E, \qquad 3V = 2E,$$

give

$$\boxed{V = 20 + 2H, \quad E = 30 + 3H, \quad F = 12 + H.}$$

Equivalently, if $V = 20T$,

$$\boxed{V = 20T, \quad E = 30T, \quad F = 10T + 2, \quad P = 12, \quad H = 10(T - 1).}$$

The combinatorial curvature of a p-gon is

$$K_p = \frac{\pi}{3}(6 - p),$$

and therefore

$$\boxed{\sum_f K_f = \frac{\pi}{3} \sum_p (6 - p) f_p = 4\pi.}$$

The twelve pentagons carry the total curvature; the hexagons carry zero combinatorial curvature.

5 Rung IV: the Eisenstein–Goldberg closure ring

Set

$$\omega = \mathrm{e}^{\mathrm{i}\pi/3} = \frac{1}{2} + \mathrm{i}\frac{\sqrt{3}}{2}, \qquad \omega^2 = \omega - 1.$$

For

$$w = k + \ell\omega \in \mathbb{Z}[\omega],$$

its norm is

$$\begin{aligned} N(w) &= (k + \ell\omega)(k + \ell\bar{\omega}) \\ &= k^2 + k\ell(\omega + \bar{\omega}) + \ell^2\omega\bar{\omega} \\ &= k^2 + k\ell + \ell^2. \end{aligned}$$

Thus

$$\boxed{T(k, \ell) = k^2 + k\ell + \ell^2 = N(k + \ell\omega).}$$

Multiplication by $k + \ell\omega$ acts on the lattice basis $(1, \omega)$ through

$$M_{k,\ell} = \begin{pmatrix} k & -\ell \\ \ell & k + \ell \end{pmatrix}.$$

Indeed,

$$(k + \ell\omega)(a + b\omega) = (ka - \ell b) + (\ell a + kb + \ell b)\omega.$$

Its determinant is

$$\boxed{\det M_{k,\ell} = k^2 + k\ell + \ell^2 = T.}$$

With

$$Q_{\mathrm{hex}} = \begin{pmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & 1 \end{pmatrix},$$

one finds

$$\boxed{M_{k,\ell}^{\mathsf{T}} Q_{\mathrm{hex}} M_{k,\ell} = T Q_{\mathrm{hex}}.}$$

Thus $M_{k,\ell}/\sqrt{T}$ is orthogonal in the hexagonal metric. Its rotation angle obeys

$$\theta_{k,\ell} = \arg(k + \ell\omega) = \arctan \frac{\sqrt{3}\,\ell}{2k + \ell}.$$

For

$$x = a + b\omega, \qquad y = c + d\omega,$$
$$xy = (ac - bd) + (ad + bc + bd)\omega,$$

and

$$\boxed{N(xy) = N(x)N(y).}$$

A fixed Goldberg multiplier g therefore yields

$$w_{n+1} = gw_n \quad \Longrightarrow \quad T_{n+1} = N(g)T_n.$$

A pure power yields

$$w_{n+1} = w_n^d \quad \Longrightarrow \quad T_{n+1} = T_n^d.$$

But a shifted power does not. With

$$w = 1 + \omega, \qquad N(w) = 3, \qquad w^2 = 3\omega, \qquad c = 1,$$

we obtain

$$N(w^2 + c) = N(1 + 3\omega) = 13,$$

while

$$N(w)^2 = 9.$$

Therefore

$$\boxed{13 \neq 9.}$$

The shifted polynomial renderer is a design analogy, not literal Goldberg norm evolution.

6 Rung V: Fibonacci selection and the Light Matrix

6.1 The two-dimensional selector

Let

$$Q_F = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \begin{pmatrix} k_{n+1} \\ \ell_{n+1} \end{pmatrix} = Q_F \begin{pmatrix} k_n \\ \ell_n \end{pmatrix}, \qquad (k_0, \ell_0) = (1, 0).$$

Then

$$(k_n, \ell_n) = (F_{n+1}, F_n).$$

The characteristic polynomial is

$$\chi_{Q_F}(x) = x^2 - x - 1,$$

so

$$\boxed{\operatorname{spec}(Q_F) = \{\phi, -\phi^{-1}\}}, \qquad \phi = \frac{1 + \sqrt{5}}{2}.$$

Binet's formula gives

$$F_n = \frac{\phi^n - (-\phi^{-1})^n}{\sqrt{5}}.$$

For the projective ratio

$$r_n = \frac{k_n}{\ell_n},$$

$$r_{n+1} = 1 + \frac{1}{r_n}.$$

Since

$$\phi = 1 + \frac{1}{\phi},$$

we have the exact error update

$$\boxed{r_{n+1} - \phi = -\frac{r_n - \phi}{\phi r_n}.}$$

Near the fixed point,

$$r_{n+1} - \phi \sim -\phi^{-2}(r_n - \phi).$$

6.2 The Lorentzian Cassini form

Define

$$q(k,\ell) = k^2 - k\ell - \ell^2 = \begin{pmatrix} k & \ell \end{pmatrix} J \begin{pmatrix} k \\ \ell \end{pmatrix},$$

with

$$J = \begin{pmatrix} 1 & -\frac{1}{2} \\ -\frac{1}{2} & -1 \end{pmatrix}.$$

Then

$$\boxed{Q_F^{\mathsf{T}} J Q_F = -J.}$$

Consequently

$$q(k_{n+1}, \ell_{n+1}) = -q(k_n, \ell_n),$$

and from $q(1,0) = 1$,

$$\boxed{k_n^2 - k_n\ell_n - \ell_n^2 = (-1)^n.}$$

The null cone

$$q(k,\ell) = 0$$

has slopes

$$\frac{k}{\ell} = \phi, \qquad \frac{k}{\ell} = -\phi^{-1}.$$

Writing

$$X = k - \frac{\ell}{2}, \qquad T_L = \frac{\sqrt{5}}{2}\ell,$$

we obtain

$$q = X^2 - T_L^2.$$

Moreover, Q_F^2 is conjugate to a Lorentz boost:

$$CQ_F^2C^{-1} = \begin{pmatrix} \frac{3}{2} & \frac{\sqrt{5}}{2} \\ \frac{\sqrt{5}}{2} & \frac{3}{2} \end{pmatrix} = \begin{pmatrix} \cosh(2\log\phi) & \sinh(2\log\phi) \\ \sinh(2\log\phi) & \cosh(2\log\phi) \end{pmatrix}.$$

6.3 The quadratic lift

For

$$u_n = \begin{pmatrix} k_n^2 \\ k_n\ell_n \\ \ell_n^2 \end{pmatrix},$$

we have

$$\begin{aligned} k_{n+1}^2 &= k_n^2 + 2k_n\ell_n + \ell_n^2, \\ k_{n+1}\ell_{n+1} &= k_n^2 + k_n\ell_n, \\ \ell_{n+1}^2 &= k_n^2. \end{aligned}$$

Thus

$$\boxed{u_{n+1} = Bu_n}, \qquad B = \mathrm{Sym}^2(Q_F) = \begin{pmatrix} 1 & 2 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}.$$

Append the invariant pentagon coordinate:

$$s_n = \begin{pmatrix} k_n^2 \\ k_n\ell_n \\ \ell_n^2 \\ P_n \end{pmatrix}, \qquad P_n = 12.$$

The Light Matrix is

$$\boxed{\mathcal{M}_{\text{light}} = \begin{pmatrix} 1 & 2 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \mathrm{Sym}^2(Q_F) \oplus [1].}$$

Its characteristic polynomial is

$$\chi_{\mathcal{M}}(x) = (x-1)\det\begin{pmatrix} x-1 & -2 & -1 \\ -1 & x-1 & 0 \\ -1 & 0 & x \end{pmatrix}$$
$$= (x-1)(x+1)(x^2-3x+1).$$

Therefore

$$\boxed{\operatorname{spec}(\mathcal{M}_{\text{light}}) = \{\phi^2, 1, -1, \phi^{-2}\}.}$$

One eigenbasis is

$$v_+ = \begin{pmatrix} \phi^2 \\ \phi \\ 1 \\ 0 \end{pmatrix}, \qquad v_P = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \qquad v_A = \begin{pmatrix} -2 \\ 1 \\ 2 \\ 0 \end{pmatrix}, \qquad v_- = \begin{pmatrix} \phi^{-2} \\ -\phi^{-1} \\ 1 \\ 0 \end{pmatrix}.$$

Hence

$$\mathcal{M}v_+ = \phi^2 v_+, \quad \mathcal{M}v_P = v_P, \quad \mathcal{M}v_A = -v_A, \quad \mathcal{M}v_- = \phi^{-2}v_-.$$

The eigenvalue-1 coordinate records $P = 12$; Euler forces the invariant, while the appended block encodes it. The matrix does not derive $P = 12$ from ϕ.

6.4 The integral Lorentz metric

Set

$$\Gamma = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & -1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Then

$$\boxed{\mathcal{M}_{\text{light}}^{\mathsf{T}}\Gamma\mathcal{M}_{\text{light}} = \Gamma.}$$

The determinant is

$$\det\Gamma = -3,$$

and the signature is $(3, 1)$. For a rank-one quadratic state

$$u = (k^2, k\ell, \ell^2)^{\mathsf{T}},$$
$$\nu^{\mathsf{T}}\Gamma_3 u = (k^2 - k\ell - \ell^2)^2.$$

For the golden orbit and $P = 12$,

$$\boxed{s_n^{\mathsf{T}}\Gamma s_n = 1 + 12^2 = 145.}$$

The growing and contracting eigenvectors are null:

$$v_+^{\mathsf{T}}\Gamma v_+ = 0, \qquad v_-^{\mathsf{T}}\Gamma v_- = 0,$$

with reciprocal eigenvalues

$$\phi^2\phi^{-2} = 1.$$

6.5 The triangulation sequence

Define

$$T_n = k_n^2 + k_n\ell_n + \ell_n^2.$$

Then

$$T_n = F_{n+1}^2 + F_{n+1}F_n + F_n^2,$$

and

$$T_n : 1, 3, 7, 19, 49, 129, 337, 883, \ldots$$

The recurrence is

$$\boxed{T_{n+3} = 2T_{n+2} + 2T_{n+1} - T_n.}$$

Its characteristic polynomial factors as

$$r^3 - 2r^2 - 2r + 1 = (r+1)(r^2 - 3r + 1),$$

with roots

$$\phi^2, -1, \phi^{-2}.$$

The closed form is

$$\boxed{T_n = \frac{2}{5}\left(\phi^{2n+2} + \phi^{-2n-2}\right) - \frac{1}{5}(-1)^n.}$$

The generating function is

$$\boxed{\sum_{n\geq 0} T_n x^n = \frac{1 + x - x^2}{1 - 2x - 2x^2 + x^3}.}$$

As $n \to \infty$,

$$\frac{T_{n+1}}{T_n} \to \phi^2, \qquad \frac{\sqrt{T_{n+1}}}{\sqrt{T_n}} \to \phi.$$

This is a sequence of independently closed shells. A fixed exact Goldberg multiplier cannot have linear ratio ϕ, because its area multiplier is an integer norm while ϕ^2 is irrational.

7 Rung VI: planar symmetry no-go and the quaternionic fork

For even N, the real cyclic group algebra decomposes as

$$\boxed{\mathbb{R}[C_N] \cong \mathbb{R}^2 \oplus \mathbb{C}^{(N-2)/2}.}$$

For the dihedral group of order $2N$,

$$\boxed{\mathbb{R}[D_N] \cong \mathbb{R}^4 \oplus M_2(\mathbb{R})^{N/2-1}.}$$

At $N = 14$,

$$\mathbb{R}[C_{14}] \cong \mathbb{R}^2 \oplus \mathbb{C}^6, \qquad \mathbb{R}[D_{14}] \cong \mathbb{R}^4 \oplus M_2(\mathbb{R})^6.$$

The Standard-Model finite algebra is

$$\boxed{A_F = \mathbb{C} \oplus \mathbb{H} \oplus M_3(\mathbb{C}).}$$

Therefore

$$\boxed{\mathbb{R}[C_{14}] \ncong A_F, \qquad \mathbb{R}[D_{14}] \ncong A_F.}$$

Every finite Euclidean point group in the plane is cyclic or dihedral, so no raw planar ornament group algebra has the required quaternionic and $M_3(\mathbb{C})$ blocks.

The order-eight fork is instructive:

$$\mathbb{R}[D_4] \cong \mathbb{R}^4 \oplus M_2(\mathbb{R}),$$

whereas

$$\boxed{\mathbb{R}[Q_8] \cong \mathbb{R}^4 \oplus \mathbb{H}.}$$

An eightfold image harmonic does not imply Q_8; a quaternionic or spinorial lift is extra structure.

8 Rung VII: the Schur commutant

Let a finite group G act on a real representation

$$H \cong \bigoplus_\alpha V_\alpha \otimes_{\mathbb{D}_\alpha} \mathbb{D}_\alpha^{m_\alpha},$$

where

$$\mathbb{D}_\alpha = \mathrm{End}_G(V_\alpha) \in \{\mathbb{R}, \mathbb{C}, \mathbb{H}\}.$$

Real Schur theory gives

$$\boxed{\mathrm{End}_G(H) \cong \bigoplus_\alpha M_{m_\alpha}(\mathbb{D}_\alpha).}$$

Choose three inequivalent isotypic sectors with

$$(\mathbb{D}_1, m_1) = (\mathbb{C}, 1), \qquad (\mathbb{D}_2, m_2) = (\mathbb{H}, 1), \qquad (\mathbb{D}_3, m_3) = (\mathbb{C}, 3).$$

Then

$$\boxed{\mathrm{End}_G(H) \cong M_1(\mathbb{C}) \oplus M_1(\mathbb{H}) \oplus M_3(\mathbb{C}) = A_F.}$$

A concrete conditional witness uses

$$G = C_{14} \times Q_8,$$

$$H_{\mathrm{wit}} = V_1 \oplus W \oplus (V_3 \otimes \mathbb{R}^3),$$

where V_1, V_3 are inequivalent real complex-type C_{14} representations and W is a quaternionic-type Q_8 representation. The real dimension is

$$2 + 4 + 6 = 12.$$

The commutant equation for generators G_j is

$$XG_j - G_jX = 0.$$

After vectorization,

$$\left(G_j^{\mathsf{T}} \otimes I - I \otimes G_j\right) \operatorname{vec} X = 0.$$

The stacked finite system has shape

$$432 \times 144,$$

rank 120, and nullity 24:

$$\boxed{\dim_{\mathbb{R}} \operatorname{Comm}_G(H_{\mathrm{wit}}) = 24 = 2 + 4 + 18.}$$

The numerical span residual against the explicit $\mathbb{C} \oplus \mathbb{H} \oplus M_3(\mathbb{C})$ basis is below 3.9×10^{-15}.

Add a trivial real line:

$$H_{\mathrm{wit}}^{+} = \mathbb{R}_{\mathrm{triv}} \oplus H_{\mathrm{wit}}.$$

Then, provided no other sector contains the trivial representation,

$$\boxed{\operatorname{End}_G(H_{\mathrm{wit}}^{+}) \cong \mathbb{R} \oplus A_F.}$$

The corresponding 13-dimensional numerical witness has commutant dimension 25 and span residual below 5.3×10^{-15}.

CONDITIONAL The Schur conclusion is exact after the division-algebra types and multiplicities are supplied. The Corinth image supplies the finite harmonic inspiration, not the Q_8 lift, the complex Schur types, or the multiplicity three.

9 Rung VIII: the finite real spectral-triple layer

A finite even real spectral triple is data

$$\mathcal{T}_F = (A_F, H_F, \pi_F, D_F, J_F, \gamma_F)$$

with

$$D_F = D_F^*, \qquad D_F\gamma_F + \gamma_F D_F = 0,$$

and KO-dimension-six signs

$$J_F^2 = +1, \qquad J_F D_F = D_F J_F, \qquad J_F\gamma_F = -\gamma_F J_F.$$

The opposite representation is

$$\pi_F^{\mathrm{op}}(b) = J_F \pi_F(b)^* J_F^{-1}.$$

The order-zero and order-one conditions are

$$[\pi_F(a), \pi_F^{\mathrm{op}}(b)] = 0,$$

$$\boxed{[[D_F, \pi_F(a)], \pi_F^{\mathrm{op}}(b)] = 0.}$$

The supplied manuscript uses

$$H_F = \mathbb{C}^{32} = \operatorname{span}\{|a, s, w, c\rangle\},$$

with

$$a, s, w \in \{\pm 1\}, \qquad c \in \{1, 2, 3, 4\},$$

$$\gamma|a, s, w, c\rangle = (as)|a, s, w, c\rangle,$$

$$J|a, s, w, c\rangle = |-a, s, w, c\rangle \quad \text{followed by complex conjugation.}$$

9.1 Exact derivation of the 272-dimensional base

Order the basis by chirality, with sixteen $\gamma = +1$ states followed by sixteen $\gamma = -1$ states. Oddness forces

$$D = \begin{pmatrix} 0 & B \\ C & 0 \end{pmatrix}.$$

Hermiticity gives

$$C = B^\dagger.$$

The reality condition $DJ = JD$ gives

$$B^{\mathsf{T}} = B.$$

Therefore B is complex symmetric. The number of independent complex entries is

$$\frac{16(16+1)}{2} = 136.$$

Each complex coordinate has two real directions, so

$$\boxed{\dim_{\mathbb{R}} \mathcal{D}_{\text{base}} = 2 \times 136 = 272.}$$

This is an exact count, not an SVD output.

9.2 The order-one map as a linear operator

Fix algebra elements a, b and define

$$\mathscr{O}_{a,b} : \mathcal{D}_{\text{base}} \to \text{End}_{\mathbb{C}}(H_F),$$
$$\mathscr{O}_{a,b}(D) = [[D, \pi(a)], \pi^{\text{op}}(b)].$$

Choose a real basis $D_1, \ldots, D_{272}$. Writing

$$D = \sum_{j=1}^{272} x_j D_j,$$

we obtain a real linear system

$$X_{a,b} x = 0.$$

For an algebra basis $\{a_\mu\}$, the admissible space is

$$\boxed{\mathcal{D}(A) = \bigcap_{\mu,\nu} \ker X_{a_\mu, a_\nu}.}$$

9.3 The source-reported table

The supplied manuscript reports

A	$\dim_{\mathbb{R}} \mathcal{D}(A)$
unit only	272
$\mathbb{C}$	146
$\mathbb{H}$	146
$\mathbb{C} \oplus \mathbb{H}$	80
$M_3(\mathbb{C})$	128
$\mathbb{C} \oplus M_3(\mathbb{C})$	16
$\mathbb{H} \oplus M_3(\mathbb{C})$	16
$\mathbb{C} \oplus \mathbb{H} \oplus M_3(\mathbb{C})$	16
$\mathbb{H}_L \oplus \mathbb{H}_R \oplus M_4(\mathbb{C})$	8.

It also reports that the three 16-dimensional spaces coincide as subspaces and that the 8-dimensional Pati–Salam space is the diagonal quark–lepton matched subspace.

10 Rung IX: the source specification fork

The standard real dimensions are

$$\dim_{\mathbb{R}} \mathbb{C} = 2, \qquad \dim_{\mathbb{R}} \mathbb{H} = 4, \qquad \dim_{\mathbb{R}} M_3(\mathbb{C}) = 18,$$

so

$$\boxed{\dim_{\mathbb{R}} A_F = 24.}$$

The displayed source dimension column instead reads

$$3, 5, 7, 19, 21, 23, 25,$$

which is systematically one larger than the standard dimensions. This may be a basis-counting convention, an implementation convention, or evidence of an additional represented scalar; the PDF alone does not decide among these possibilities.

The prose also states that the anti-particle $M_3(\mathbb{C})$ action fixes the lepton colour by the identity. If interpreted literally as

$$\pi(\lambda, q, m)|_{\bar{\ell}} = I \quad \text{for every } (\lambda, q, m),$$

then

$$\pi(0)|_{\bar{\ell}} = I \neq 0,$$

so the assignment is affine rather than linear. The numerical reconstruction gives

$$\|\pi(0)\|_F = 2, \qquad \|\pi(a+b) - \pi(a) - \pi(b)\|_F = 2.$$

One source-aligned linear completion introduces an independent real scalar r on the anti-lepton block:

$$\boxed{A_F^{\mathrm{src}+} = \mathbb{R}_{\bar{\ell}} \oplus \mathbb{C} \oplus \mathbb{H} \oplus M_3(\mathbb{C}).}$$

Its real dimension is

$$1 + 2 + 4 + 18 = 25.$$

Within this reconstructed representation, every row of the displayed table is reproduced. This is a conditional model of the source computation, not proof that the intended published algebra is $\mathbb{R} \oplus A_F$.

A second trial ties the anti-lepton scalar to $\lambda \in \mathbb{C}$, retaining standard A_F. In the same simplified (H_F, J, γ) implementation it yields

$$\boxed{\dim_{\mathbb{R}} \mathcal{D}(\pi_{\mathrm{trial}}(A_F)) \doteq 32,}$$

with residual 4.5×10^{-15}. This is a representation stress test, not a theorem about all noncommutative-geometric Standard-Model conventions.

11 Rung X: exact finite-field rank sandwiches

Let

$$X \in M_{m \times 272}(\mathbb{Z})$$

be a stacked integer order-one constraint matrix. If

$$\operatorname{rank}_{\mathbb{F}_p}(X \bmod p) = r,$$

then some $r \times r$ minor is nonzero modulo p, hence nonzero as an integer. Therefore

$$\operatorname{rank}_{\mathbb{Q}}(X) \geq r, \qquad \operatorname{nullity}_{\mathbb{Q}}(X) \leq 272 - r.$$

If d linearly independent exact null vectors are exhibited, then

$$\operatorname{nullity}_{\mathbb{Q}}(X) \geq d.$$

When

$$d = 272 - r,$$

the bounds meet and the dimension is exact.

11.1 Source-aligned unit-completed model

For the reconstructed $A_F^{\mathrm{src}+}$ action, deterministic integer constraints give

$$\operatorname{rank}_{\mathbb{F}_p} X_{\mathrm{src}} = 256$$

for each tested prime

$$p \in \{1009, 1013, 10007, 10009, 65519, 65521\}$$

and for three independent deterministic witness seeds. Thus

$$\text{nullity} \le 16.$$

Sixteen explicit physical Yukawa directions satisfy every algebra-basis order-one constraint with exact entrywise zero residual and have real rank 16. Therefore

$$\boxed{\dim_{\mathbb{R}} \mathcal{D}(A_F^{\mathrm{src}+}) = 16}$$

within the declared reconstructed representation.

11.2 Pati–Salam model

For

$$A_{PS} = \mathbb{H}_L \oplus \mathbb{H}_R \oplus M_4(\mathbb{C}),$$

the deterministic modular ranks are

$$\operatorname{rank}_{\mathbb{F}_p} X_{PS} = 264$$

for the same six primes and three seeds. Hence

$$\text{nullity} \le 8.$$

Eight exact paired Yukawa directions satisfy every basis constraint and have rank 8, so

$$\boxed{\dim_{\mathbb{R}} \mathcal{D}(A_{PS}) = 8.}$$

The paired directions are

$$Y^{PS}_{w_L w_R, \epsilon} = Y^{q}_{w_L w_R, \epsilon} + Y^{\ell}_{w_L w_R, \epsilon},$$

with

$$w_L, w_R \in \{+, -\}, \qquad \epsilon \in \{1, \mathrm{i}\},$$

giving $2 \times 2 \times 2 = 8$ real directions.

12 Rung XI: the relative selector and its boundary

On a fixed, unambiguous Standard-Model bimodule, one may define

$$d(A) = \dim_{\mathbb{R}} \mathcal{D}(A),$$

and indicators

$$c_3(A) = \begin{cases} 1, & \mathfrak{su}(3) \text{ acts on the quark triplet,} \\ 0, & \text{otherwise,} \end{cases}$$

$$c_2(A) = \begin{cases} 1, & \mathfrak{su}(2)_L \text{ acts chirally on left doublets,} \\ 0, & \text{otherwise,} \end{cases}$$

$$s_R(A) = \begin{cases} 1, & \exists Z \in \mathfrak{z}(\mathfrak{u}(A)) : \rho_{u_R}(Z) \neq \rho_{d_R}(Z), \\ 0, & \text{otherwise.} \end{cases}$$

A lexicographic relative selector is

$$\Delta(A) = \big(\mathbf{1}_{d(A)\neq 16}, 1 - c_3(A), 1 - c_2(A), 1 - s_R(A), \dim_{\mathbb{R}} A\big)\,.$$

For a finite candidate domain and B larger than every algebra dimension, the scalar penalty

$$\mathcal{F}_{\mathrm{rel}}(A) = B^4 \mathbf{1}_{d(A)\neq 16} + B^3(1 - c_3(A)) + B^2(1 - c_2(A)) + B(1 - s_R(A)) + \dim_{\mathbb{R}} A$$

encodes the same ordering.

However, this is a classifier after the physical sector labels and the represented algebra have already been supplied. In the present source reconstruction, the 16-dimensional endpoint belongs exactly to $A_F^{\mathrm{src}+}$, while a trial standard-A_F completion gives a different computed value. Therefore

$$\boxed{\text{relative Step 4 is conditional on representation repair.}}$$

13 Rung XII: global Step 4

Let

$$\mathfrak{M} = \{\text{finite KO-6 decorated spectral triples}\}/\sim .$$

The desired theorem is the existence of a basis-independent, noncircular functional

$$\mathfrak{F} : \mathfrak{M} \to \mathbb{R}$$

with

$$\mathfrak{F}(\mathcal{T}) \geq \mathfrak{F}(\mathcal{T}_{SM})$$

for every admissible $\mathcal{T}$, and

$$\mathfrak{F}(\mathcal{T}) = \mathfrak{F}(\mathcal{T}_{SM}) \quad \Longleftrightarrow \quad \mathcal{T} \sim \mathcal{T}_{SM}.$$

A schematic spectral-free-energy candidate is

$$Z_{\beta,f}(\mathcal{T}) = \int_{\mathcal{X}_{\mathcal{T}}} \exp\left[-\beta S_f(\mathcal{T}; D, \mathbb{A})\right] \, d\mu_{\mathcal{T}}(D, \mathbb{A}),$$

$$\mathfrak{F}_{\beta,f}(\mathcal{T}) = -\frac{1}{\beta} \log Z_{\beta,f}(\mathcal{T}).$$

The measure, normalization, candidate class, uniqueness, and stability under f and β are not supplied by the present tower.

Therefore the final open rung is

$$\boxed{\arg\min_{\mathcal{T}\in\mathfrak{M}} \mathfrak{F}(\mathcal{T}) = \mathcal{T}_{SM} \qquad \text{(trivial)}}$$

with the mandatory footnote:

$$\boxed{\text{“trivial” is the joke; the mathematical status is OPEN.}}$$

14 Rung XIII: proof by kernel and the architecture receipt

The verification environment recorded by the stress harness is

architecture :	$x86\_64$,
CPU allocation :	5 virtual cores,
processor :	AMD EPYC 9V74,
memory :	5.9 GiB total, no swap,
pointer width :	64 bits,
Float64 mantissa :	53 bits,
SIMD exposed :	SSE2, AVX, AVX2, AVX-512F, FMA, BMI2, SHA.

The destructive hardware maximum was not attempted. Instead, the harness used explicit time and memory discipline and obtained:

$$\boxed{N_{\text{Python BigInt}} = 500000, \qquad \#\text{digits}(T_N) = 208988,}$$

with independent fast-doubling equality, and

$$\boxed{N_{\text{Node BigInt}} = 200000, \qquad \#\text{digits}(T_N) = 83596.}$$

The full architecture stress completed in approximately 25.1 seconds with peak resident memory approximately 245 MB. The modular-rank stress used

$$3 \text{ deterministic seeds} \times 6 \text{ primes}$$

for each of the 16- and 8-dimensional certificates. The regression suite passed

$$\boxed{8/8}$$

checks.

Why code can be perfect mathematics. A block of code is mathematical when its specification, representation, arithmetic domain, invariants, and failure conditions are explicit. The Python BigInt recurrence is exact integer arithmetic; the finite-field row reduction is exact arithmetic in $\mathbb{F}_p$; the JavaScript BigInt kernel is exact integer arithmetic. The floating commutant SVD remains COMPUTED, and its residual is printed rather than hidden.

15 Final ledger

Claim	status	receipt
IFS fixed point under contractions	EXACT	Hutchinson contraction theorem
Smooth logarithmic spiral has Hausdorff dimension 1	EXACT	smooth-curve geometry
Scale-string complex dimensions	EXACT	poles of geometric zeta function
Inner $m = 14$, outer $m = 8$	COMPUTED	finite image/annulus sweep
Literal ancient logarithmic-spiral construction	OPEN	photograph does not establish method
Fullerene $P = 12$, $\chi = 2$	EXACT	Euler and trivalence
Eisenstein norm and Goldberg counts	EXACT	integer ring identities
Shifted Multibrot equals GC refinement	false	$13 \neq 9$ counterexample
Light Matrix and spectrum	EXACT	symmetric square and characteristic polynomial
Lorentzian invariant $\mathcal{M}^T \Gamma \mathcal{M} = \Gamma$	EXACT	symbolic matrix identity
Planar group algebra equals A_F	false	cyclic/dihedral Wedderburn decomposition
Decorated commutant equals A_F	CONDITIONAL	Schur types $(\mathbb{C}, 1), (\mathbb{H}, 1), (\mathbb{C}, 3)$
Base Dirac dimension 272	EXACT	complex symmetric 16×16 block
Source-aligned unit-completed dimension 16	EXACT	finite-field rank sandwich in declared model
Pati–Salam dimension 8	EXACT	finite-field rank sandwich in declared model
Standard-A_F trial dimension 32	COMPUTED	one representation completion only
Relative Step 4	OPEN	representation fork unresolved
Global Step 4	TRIVIAL	joke label; mathematically open

A Code block A: architecture-bounded exact and modular verifier

```python
#!/usr/bin/env python3
"""SOL FABLE LaTeX Tower v2.0 - architecture-bounded verification.

This is a deterministic stress harness, not an attempt to crash the host.
It records the available CPU/memory/SIMD envelope, executes exact arbitrary-
precision recurrence checks to a declared depth, repeats the finite-field rank
certificates across independent deterministic witnesses, validates the Schur
commutant, and executes a browser-style BigInt kernel under Node.

Status grammar:
  EXACT       symbolic/integer/finite-field proof within the declared model.
  COMPUTED    finite-precision result with residual and environment receipt.
  CONDITIONAL exact after explicit representation assumptions.
  TRIVIAL     the traditional mathematician's joke for the still-open rung.
"""
from __future__ import annotations

import hashlib
import json
import os
import platform
import resource
import subprocess
import sys
import time
from pathlib import Path
from typing import Dict, Iterable, Tuple

import numpy as np
import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
RECEIPTS = ROOT / "receipts"
RECEIPTS.mkdir(parents=True, exist_ok=True)

# Import the complete v1 reconstruction kernel copied into this bundle.
import sol_fable_tower_audit_v1 as A

if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)


def sha256_bytes(data: bytes) -> str:
```

```
    return hashlib.sha256(data).hexdigest()


def fib_pair(n: int) -> Tuple[int, int]:
    """Return (F_n, F_{n+1}) by exact fast doubling."""
    if n < 0:
        raise ValueError("n must be nonnegative")
    if n == 0:
        return 0, 1
    a, b = fib_pair(n >> 1)
    c = a * ((b << 1) - a)
    d = a * a + b * b
    return (d, c + d) if (n & 1) else (c, d)


def exact_triangulation(n: int) -> int:
    """T_n = F_{n+1}^2 + F_{n+1}F_n + F_n^2."""
    fn, fn1 = fib_pair(n)
    return fn1 * fn1 + fn1 * fn + fn * fn


def architecture_info() -> Dict[str, object]:
    cpu_flags = []
    model_name = "unknown"
    try:
        for line in Path("/proc/cpuinfo").read_text().splitlines():
            if line.startswith("model name") and model_name == "unknown":
                model_name = line.split(":", 1)[1].strip()
            elif line.startswith("flags") and not cpu_flags:
                cpu_flags = line.split(":", 1)[1].split()
    except OSError:
        pass

    mem_total = None
    mem_available = None
    try:
        mem = {}
        for line in Path("/proc/meminfo").read_text().splitlines():
            key, value = line.split(":", 1)
            mem[key] = int(value.strip().split()[0]) * 1024
        mem_total = mem.get("MemTotal")
        mem_available = mem.get("MemAvailable")
    except OSError:
        pass

    return {
        "platform": platform.platform(),
        "machine": platform.machine(),
        "python": sys.version,
        "cpu_count": os.cpu_count(),
        "cpu_model": model_name,
        "byteorder": sys.byteorder,
        "pointer_bits": 8 * np.dtype(np.intp).itemsize,
        "float64_mantissa_bits": np.finfo(np.float64).nmant + 1,
        "float64_epsilon": float(np.finfo(np.float64).eps),
        "memory_total_bytes": mem_total,
        "memory_available_bytes_at_start": mem_available,
        "simd_flags_of_interest": [
            f for f in ("sse2", "avx", "avx2", "avx512f", "fma", "bmi2", "sha_ni")
            if f in cpu_flags
        ],
    }


def stress_exact_ladder(depth: int = 500_000) -> Dict[str, object]:
    """Advance the exact T recurrence to a large but non-destructive depth.

    The recurrence is not accepted merely because it generated its own output:
    the terminal value is independently recomputed from Fibonacci fast doubling.
    """
    if depth < 3:
        raise ValueError("depth must be at least 3")
    t0 = time.perf_counter()
    t_nm2, t_nm1, t_n = 1, 3, 7
    for _ in range(3, depth + 1):
        t_np1 = 2 * t_n + 2 * t_nm1 - t_nm2
        t_nm2, t_nm1, t_n = t_nm1, t_n, t_np1
    recurrence_seconds = time.perf_counter() - t0

    t1 = time.perf_counter()
    independent = exact_triangulation(depth)
    fast_doubling_seconds = time.perf_counter() - t1
    if independent != t_n:
        raise AssertionError("recurrence and fast-doubling values disagree")

    # Exact modular digests avoid storing the huge decimal string in the receipt.
    primes = (1_000_000_007, 1_000_000_009, 2_147_483_647)
    residues = {str(p): int(t_n % p) for p in primes}
    digits = len(str(t_n))
    byte_len = (t_n.bit_length() + 7) // 8
    digest = sha256_bytes(t_n.to_bytes(byte_len, "big"))

```

```
    return {
        "depth": depth,
        "terminal_decimal_digits": digits,
        "terminal_bit_length": t_n.bit_length(),
        "terminal_sha256_big_endian": digest,
        "terminal_prime_residues": residues,
        "recurrence_seconds": recurrence_seconds,
        "fast_doubling_crosscheck_seconds": fast_doubling_seconds,
        "exact_match": True,
    }


def fixed_extended_element(seed: int, slot: int):
    rng = np.random.default_rng(seed + 104729 * slot)
    return A.random_extended_elem(rng)


def fixed_ps_element(seed: int, slot: int):
    rng = np.random.default_rng(seed + 130363 * slot)
    return A.random_ps_elem(rng)


def repeated_modular_rank_receipt() -> Dict[str, object]:
    """Repeat exact rank lower bounds with independent deterministic witnesses.

    Each rank is computed modulo a prime on an integer constraint matrix.  A
    rank-r minor nonzero mod p is a nonzero integer minor, hence rank_Q >= r.
    The explicit null vectors checked against the full algebra basis supply the
    opposite nullity inequality.
    """
    primes = (1009, 1013, 10007, 10009, 65519, 65521)
    seeds = (20260803, 31415926, 27182818)
    ext_runs = []
    ps_runs = []

    t0 = time.perf_counter()
    for seed in seeds:
        ext_parts = []
        for slot in range(3):
            left = A.pi_extended(fixed_extended_element(seed, 2 * slot))
            right = A.opposite(A.pi_extended(fixed_extended_element(seed, 2 * slot + 1)))
            ext_parts.append(A.constraint_matrix(A.DBASE, left, right))
        x_ext = np.concatenate(ext_parts, axis=0)
        ranks_ext = {str(p): A.rank_mod(x_ext, p) for p in primes}
        ext_runs.append({"seed": seed, "shape": list(x_ext.shape), "ranks": ranks_ext})

        left_ps = A.pi_ps(fixed_ps_element(seed, 0))
        right_ps = A.opposite(A.pi_ps(fixed_ps_element(seed, 1)))
        x_ps = A.constraint_matrix(A.DBASE, left_ps, right_ps)
        ranks_ps = {str(p): A.rank_mod(x_ps, p) for p in primes}
        ps_runs.append({"seed": seed, "shape": list(x_ps.shape), "ranks": ranks_ps})

    y16 = A.physical_yukawa_basis()
    y8 = A.paired_ps_yukawa_basis()
    y16_rank = A.real_span_rank(y16)
    y8_rank = A.real_span_rank(y8)
    y16_residual = A.max_order_one_residual(y16, A.extended_basis("CHM"))
    y8_residual = A.max_order_one_residual(y8, A.ps_basis())

    ext_all = [r for run in ext_runs for r in run["ranks"].values()]
    ps_all = [r for run in ps_runs for r in run["ranks"].values()]
    exact16 = set(ext_all) == {256} and y16_rank == 16 and y16_residual == 0.0
    exact8 = set(ps_all) == {264} and y8_rank == 8 and y8_residual == 0.0
    if not (exact16 and exact8):
        raise AssertionError("rank sandwich stress failed")

    return {
        "primes": list(primes),
        "seeds": list(seeds),
        "extended_runs": ext_runs,
        "pati_salam_runs": ps_runs,
        "explicit_yukawa16_rank": y16_rank,
        "explicit_yukawa16_full_basis_residual": y16_residual,
        "explicit_paired8_rank": y8_rank,
        "explicit_paired8_full_basis_residual": y8_residual,
        "exact_extended_nullity": 16,
        "exact_pati_salam_nullity": 8,
        "seconds": time.perf_counter() - t0,
    }


def exact_symbolic_receipt() -> Dict[str, object]:
    q = sp.Matrix([[1, 1], [1, 0]])
    b = A.symmetric_square_2x2(q)
    m = sp.diag(1, 1, 1, 1)
    m[:3, :3] = b
    lam = sp.symbols("lambda")
    gamma = sp.Matrix([
        [1, -1, 0, 0],
        [-1, -1, 1, 0],
        [0, 1, 1, 0],
        [0, 0, 0, 1],
```

```
    ])
    j = sp.Matrix([[1, sp.Rational(-1, 2)], [sp.Rational(-1, 2), -1]])
    checks = {
        "sym2": [[int(b[i, j_]) for j_ in range(3)] for i in range(3)],
        "charpoly_Q": str(sp.factor(q.charpoly(lam).as_expr())),
        "charpoly_M": str(sp.factor(m.charpoly(lam).as_expr())),
        "Q_T_J_Q_equals_minus_J": bool(q.T * j * q == -j),
        "M_T_Gamma_M_equals_Gamma": bool(m.T * gamma * m == gamma),
        "Gamma_det": int(gamma.det()),
        "Gamma_eigenvalue_signs_numeric": [
            int(np.sign(x)) for x in np.linalg.eigvalsh(np.array(gamma, dtype=float))
        ],
    }
    if checks["charpoly_M"] != "(lambda - 1)*(lambda + 1)*(lambda**2 - 3*lambda + 1)":
        raise AssertionError("unexpected Light Matrix characteristic polynomial")
    return checks


def node_bigint_receipt(depth: int = 200_000) -> Dict[str, object]:
    js = ROOT / "code" / "sol_tower_browser_v2.js"
    proc = subprocess.run(
        ["node", str(js), str(depth)],
        cwd=ROOT,
        check=True,
        capture_output=True,
        text=True,
    )
    result = json.loads(proc.stdout)
    if not result.get("exact_match"):
        raise AssertionError("Node BigInt cross-check failed")
    return result


def memory_peak_bytes() -> int:
    value = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    # Linux reports KiB; macOS reports bytes.
    return int(value * 1024 if sys.platform.startswith("linux") else value)


def main() -> None:
    start = time.perf_counter()
    receipt: Dict[str, object] = {
        "version": "SOL_FABLE_LATEX_TOWER_v2.0",
        "status_boundary": {
            "exact": "symbolic, integer, or finite-field certificate within declared model",
            "computed": "finite arithmetic with residual and environment",
            "conditional": "exact only after explicit representation assumptions",
            "trivial": "mathematical joke label for the still-open global Step 4",
        },
        "architecture": architecture_info(),
        "symbolic": exact_symbolic_receipt(),
        "exact_ladder_stress": stress_exact_ladder(),
        "modular_rank_stress": repeated_modular_rank_receipt(),
        "commutant": A.extended_commutant_receipt(),
        "node_bigint": node_bigint_receipt(),
    }
    receipt["elapsed_seconds"] = time.perf_counter() - start
    receipt["peak_rss_bytes"] = memory_peak_bytes()
    out = RECEIPTS / "sol_architecture_stress_v2.json"
    out.write_text(json.dumps(receipt, indent=2, sort_keys=True) + "\n", encoding="utf-8")
    print(json.dumps({
        "receipt": str(out),
        "elapsed_seconds": receipt["elapsed_seconds"],
        "peak_rss_bytes": receipt["peak_rss_bytes"],
        "ladder_depth": receipt["exact_ladder_stress"]["depth"],
        "ladder_digits": receipt["exact_ladder_stress"]["terminal_decimal_digits"],
        "extended_nullity": receipt["modular_rank_stress"]["exact_extended_nullity"],
        "pati_salam_nullity": receipt["modular_rank_stress"]["exact_pati_salam_nullity"],
        "node_depth": receipt["node_bigint"]["depth"],
    }, indent=2))


if __name__ == "__main__":
    main()
```

B Code block B: browser/Node BigInt kernel

```
#!/usr/bin/env node
"use strict";

// Browser/Node BigInt kernel for the exact golden T recurrence.
// No floating-point arithmetic is used for the recurrence or topology checks.
const depth = Number(process.argv[2] || 200000);
if (!Number.isSafeInteger(depth) || depth < 3 || depth > 1000000) {
  throw new Error("depth must be a safe integer in [3, 1000000]");
}

function fibPair(n) {
```

```javascript
  if (n === 0n) return [0n, 1n];
  const [a, b] = fibPair(n >> 1n);
  const c = a * ((b << 1n) - a);
  const d = a * a + b * b;
  return (n & 1n) ? [d, c + d] : [c, d];
}

function exactT(n) {
  const [fn, fn1] = fibPair(BigInt(n));
  return fn1 * fn1 + fn1 * fn + fn * fn;
}

const t0 = process.hrtime.bigint();
let tm2 = 1n, tm1 = 3n, tn = 7n;
for (let n = 3; n <= depth; n++) {
  const next = 2n * tn + 2n * tm1 - tm2;
  tm2 = tm1; tm1 = tn; tn = next;
}
const recurrenceSeconds = Number(process.hrtime.bigint() - t0) / 1e9;
const independent = exactT(depth);
const exactMatch = independent === tn;
if (!exactMatch) throw new Error("recurrence mismatch");

// Euler is checked exactly on the terminal shell.
const V = 20n * tn;
const E = 30n * tn;
const F = 10n * tn + 2n;
const chi = V - E + F;
if (chi !== 2n) throw new Error("Euler closure failed");

const text = tn.toString();
const result = {
  depth,
  terminal_decimal_digits: text.length,
  recurrence_seconds: recurrenceSeconds,
  exact_match: exactMatch,
  euler_characteristic: chi.toString(),
  residues: {
    "1000000007": (tn % 1000000007n).toString(),
    "1000000009": (tn % 1000000009n).toString()
  }
};
process.stdout.write(JSON.stringify(result));
```

C Code block C: regression suite

```python
#!/usr/bin/env python3
from __future__ import annotations

import json
import sys
from pathlib import Path

import sympy as sp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
sys.path.insert(0, str(HERE))

import sol_fable_tower_audit_v1 as A
import sol_architecture_stress_v2 as S

R = json.loads((ROOT / "receipts" / "sol_architecture_stress_v2.json").read_text())


def test_light_matrix_exact():
    q = sp.Matrix([[1, 1], [1, 0]])
    assert A.symmetric_square_2x2(q) == sp.Matrix([[1, 2, 1], [1, 1, 0], [1, 0, 0]])
    assert R["symbolic"]["charpoly_M"] == "(lambda - 1)*(lambda + 1)*(lambda**2 - 3*lambda + 1)"
    assert R["symbolic"]["M_T_Gamma_M_equals_Gamma"]


def test_architecture_receipt():
    a = R["architecture"]
    assert a["machine"] == "x86_64"
    assert a["pointer_bits"] == 64
    assert a["float64_mantissa_bits"] == 53


def test_big_integer_depth():
    x = R["exact_ladder_stress"]
    assert x["depth"] == 500_000
    assert x["terminal_decimal_digits"] == 208_988
    assert x["exact_match"]
    assert len(x["terminal_sha256_big_endian"]) == 64


def test_node_bigint():
```

```
    x = R["node_bigint"]
    assert x["depth"] == 200_000
    assert x["terminal_decimal_digits"] == 83_596
    assert x["exact_match"]
    assert x["euler_characteristic"] == "2"


def test_modular_rank_stress():
    x = R["modular_rank_stress"]
    assert x["exact_extended_nullity"] == 16
    assert x["exact_pati_salam_nullity"] == 8
    for run in x["extended_runs"]:
        assert set(run["ranks"].values()) == {256}
    for run in x["pati_salam_runs"]:
        assert set(run["ranks"].values()) == {264}
    assert x["explicit_yukawa16_rank"] == 16
    assert x["explicit_yukawa16_full_basis_residual"] == 0.0
    assert x["explicit_paired8_rank"] == 8
    assert x["explicit_paired8_full_basis_residual"] == 0.0


def test_commutant():
    x = R["commutant"]
    assert x["base_AF_witness"]["computed_commutant_dimension"] == 24
    assert x["base_AF_witness"]["verdict"] == "PASS"
    assert x["computed_commutant_dimension"] == 25
    assert x["verdict"] == "PASS"


def test_genesis_counterexample():
    seed = (1, 1)
    squared = A.eis_mul(seed, seed)
    shifted = (squared[0] + 1, squared[1])
    assert A.eis_norm(seed) == 3
    assert A.eis_norm(shifted) == 13
    assert A.eis_norm(seed) ** 2 == 9


def test_open_rung_is_joke_not_result():
    assert R["status_boundary"]["trivial"].startswith("mathematical joke")


def main():
    tests = [value for name, value in sorted(globals().items()) if name.startswith("test_")]
    for test in tests:
        test()
        print("PASS", test.__name__)
    print(f"ALL {len(tests)}/{len(tests)} PASS")


if __name__ == "__main__":
    main()
```

D Code block D: complete finite-bimodule and commutant reconstruction kernel

```
#!/usr/bin/env python3
"""SOL FABLE LaTeX Tower v1.0 - exact/computed audit.

This program unifies four layers that were separate in the source artifacts:
  1. the Eisenstein/Goldberg parameter plane and the Light Matrix;
  2. the Corinth-mosaic angular-mode audit and Schur-commutant witness;
  3. the finite KO-dimension-6 bimodule calculation described in the supplied PDF;
  4. a specification audit of the phrase "the lepton colour is fixed (identity action)".

Status grammar:
  EXACT       symbolic/integer proof or an exact rank sandwich;
  COMPUTED    finite-precision result with a residual/tolerance;
  CONDITIONAL exact after declared representation assumptions;
  OPEN        not selected/derived by the calculation.

The script is intentionally deterministic. No network access and no random claims.
"""
from __future__ import annotations

import contextlib
import hashlib
import importlib.util
import json
import math
import os
import re
import shutil
import subprocess
import sys
import time
from dataclasses import dataclass
```

```python
from pathlib import Path
from typing import Dict, Iterable, List, Sequence, Tuple

import numpy as np
import scipy.linalg as la
import sympy as sp

ROOT = Path(__file__).resolve().parent
FIG = ROOT / "figures"
REC = ROOT / "receipts"
SRC = ROOT / "source_inputs"
FIG.mkdir(exist_ok=True)
REC.mkdir(exist_ok=True)

PHI = (1.0 + math.sqrt(5.0)) / 2.0


def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def json_dump(path: Path, obj: object) -> None:
    path.write_text(json.dumps(obj, indent=2, sort_keys=True) + "\n", encoding="utf-8")


def run_fable_image_audit() -> Dict[str, object]:
    """Run the frozen FABLE v0.2 audit in this bundle."""
    script = ROOT / "fable_mosaic_audit_v0_2.py"
    proc = subprocess.run(
        [sys.executable, str(script)], cwd=ROOT, capture_output=True, text=True, check=True
    )
    (REC / "fable_v0_2_stdout.txt").write_text(proc.stdout, encoding="utf-8")
    receipt_path = ROOT / "fable_v0_2_receipt.json"
    receipt = json.loads(receipt_path.read_text(encoding="utf-8"))
    for name in ("fable_mode_spectrum.png", "fable_polar_strip.png"):
        p = ROOT / name
        if p.exists():
            shutil.copy2(p, FIG / name)
    shutil.copy2(receipt_path, REC / receipt_path.name)
    shutil.copy2(ROOT / "fable_v0_2_results.txt", REC / "fable_v0_2_results.txt")
    return receipt


# ---------------------------------------------------------------------------
# I. Light Matrix / Eisenstein plane
# ---------------------------------------------------------------------------

def symmetric_square_2x2(Q: sp.Matrix) -> sp.Matrix:
    a, b, c, d = Q[0, 0], Q[0, 1], Q[1, 0], Q[1, 1]
    return sp.Matrix(
        [
            [a * a, 2 * a * b, b * b],
            [a * c, a * d + b * c, b * d],
            [c * c, 2 * c * d, d * d],
        ]
    )


def eis_norm(pair: Tuple[int, int]) -> int:
    k, l = pair
    return k * k + k * l + l * l


def eis_mul(x: Tuple[int, int], y: Tuple[int, int]) -> Tuple[int, int]:
    """Multiply k+l*w with w=e^{i*pi/3}, w^2=w-1."""
    a, b = x
    c, d = y
    return a * c - b * d, a * d + b * c + b * d


def light_matrix_receipt(depth: int = 1000) -> Dict[str, object]:
    Q = sp.Matrix([[1, 1], [1, 0]])
    B = symmetric_square_2x2(Q)
    M = sp.diag(1, 1, 1, 1)
    M[:3, :3] = B

    lam = sp.symbols("lambda")
    char_Q = sp.factor(Q.charpoly(lam).as_expr())
    char_M = sp.factor(M.charpoly(lam).as_expr())

    J2 = sp.Matrix([[1, sp.Rational(-1, 2)], [sp.Rational(-1, 2), -1]])
    Gamma4 = sp.Matrix(
        [
            [1, -1, 0, 0],
            [-1, -1, 1, 0],
            [0, 1, 1, 0],
            [0, 0, 0, 1],
        ]
```

```
    )

    # Big-integer ladder: no Float64 truncation.
    k, l = 1, 0
    pairs: List[Tuple[int, int]] = []
    Ts: List[int] = []
    cassini: List[int] = []
    for n in range(depth + 1):
        pairs.append((k, l))
        Ts.append(eis_norm((k, l)))
        cassini.append(k * k - k * l - l * l)
        k, l = k + l, k
    recurrence_residual = max(
        abs(Ts[n + 3] - (2 * Ts[n + 2] + 2 * Ts[n + 1] - Ts[n]))
        for n in range(depth - 2)
    )
    cassini_residual = max(abs(cassini[n] - ((-1) ** n)) for n in range(depth + 1))
    topology_residual = max(abs((20 * T) - (30 * T) + (10 * T + 2) - 2) for T in Ts)

    # Norm multiplicativity over a deterministic integer box.
    mult_residual = 0
    for a in range(-5, 6):
        for b in range(-5, 6):
            for c in range(-3, 4):
                for d in range(-3, 4):
                    lhs = eis_norm(eis_mul((a, b), (c, d)))
                    rhs = eis_norm((a, b)) * eis_norm((c, d))
                    mult_residual = max(mult_residual, abs(lhs - rhs))

    # Exact correction to GENESIS-alpha: addition c breaks norm-power closure.
    seed = (1, 1)  # norm 3
    cshift = (1, 0)
    squared = eis_mul(seed, seed)  # (0,3)
    shifted = (squared[0] + cshift[0], squared[1] + cshift[1])  # (1,3)
    genesis_counterexample = {
        "seed_pair": seed,
        "seed_T": eis_norm(seed),
        "power_d": 2,
        "shift_pair": cshift,
        "next_pair": shifted,
        "actual_next_T": eis_norm(shifted),
        "claimed_power_T": eis_norm(seed) ** 2,
        "difference": eis_norm(shifted) - eis_norm(seed) ** 2,
    }

    # HTML syntax: extract inline scripts and ask Node to parse each.
    html_path = SRC / "shell__genesis_alpha_v0_1(1).html"
    html = html_path.read_text(encoding="utf-8")
    scripts = re.findall(r"<script(?:\s[^>]*)?>(.*?)</script>", html, flags=re.I | re.S)
    node_checks: List[Dict[str, object]] = []
    for i, js in enumerate(scripts):
        js_path = REC / f"genesis_inline_{i}.js"
        js_path.write_text(js, encoding="utf-8")
        p = subprocess.run(["node", "--check", str(js_path)], capture_output=True, text=True)
        node_checks.append(
            {
                "index": i,
                "returncode": p.returncode,
                "stderr": p.stderr.strip(),
                "bytes": js_path.stat().st_size,
            }
        )

    return {
        "status": "EXACT_PLUS_COMPUTED_SYNTAX",
        "Q": [[1, 1], [1, 0]],
        "Sym2_Q": [[int(B[i, j]) for j in range(3)] for i in range(3)],
        "M_light": [[int(M[i, j]) for j in range(4)] for i in range(4)],
        "charpoly_Q": str(char_Q),
        "charpoly_M": str(char_M),
        "Q_transpose_J_Q_equals_minus_J": bool(Q.T * J2 * Q == -J2),
        "M_transpose_Gamma_M_equals_Gamma": bool(M.T * Gamma4 * M == Gamma4),
        "Gamma4_determinant": int(Gamma4.det()),
        "Gamma4_signature_numeric": [3, 1],
        "depth": depth,
        "first_T": Ts[:12],
        "last_T_decimal_digits": len(str(Ts[-1])),
        "recurrence_max_integer_residual": recurrence_residual,
        "cassini_max_integer_residual": cassini_residual,
        "topology_max_integer_residual": topology_residual,
        "eisenstein_norm_multiplicativity_max_integer_residual": mult_residual,
        "genesis_alpha_counterexample": genesis_counterexample,
        "genesis_alpha_correct_domain": {
            "fixed_multiplier": "w_{n+1}=g*w_n implies T_{n+1}=N(g)T_n",
            "pure_power": "w_{n+1}=w_n^d (c=0) implies T_{n+1}=T_n^d",
            "shifted_power": "w_{n+1}=w_n^d+c has no such norm recurrence in general",
        },
        "genesis_html_sha256": sha256(html_path),
        "node_inline_script_checks": node_checks,
    }
```

```python
# ---------------------------------------------------------------------------
# II. Extend the FABLE Schur witness by a fixed real line
# ---------------------------------------------------------------------------

def load_fable_module():
    path = ROOT / "fable_mosaic_audit_v0_2.py"
    spec = importlib.util.spec_from_file_location("fable_local", path)
    if spec is None or spec.loader is None:
        raise RuntimeError("Cannot import FABLE audit module")
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod)
    return mod


def extended_commutant_receipt() -> Dict[str, object]:
    fable = load_fable_module()
    base = fable.explicit_schur_witness()

    r1 = fable.rotation(2.0 * math.pi / 14.0)
    r3 = fable.rotation(6.0 * math.pi / 14.0)
    li, lj = fable.quaternion_left_generators()
    i1, i2, i4, i6 = np.eye(1), np.eye(2), np.eye(4), np.eye(6)
    color_rot = np.kron(np.eye(3), r3)

    generators = [
        fable.block_diag(i1, r1, i4, color_rot),
        fable.block_diag(i1, i2, li, i6),
        fable.block_diag(i1, i2, lj, i6),
    ]
    n = 13
    eye = np.eye(n)
    constraints = np.vstack([np.kron(g.T, eye) - np.kron(eye, g) for g in generators])
    _, singular, vh = np.linalg.svd(constraints, full_matrices=True)
    tol = 1e-10 * singular[0]
    rank = int(np.sum(singular > tol))
    null_basis = vh.T[:, rank:]

    j2 = np.array([[0.0, -1.0], [1.0, 0.0]])
    z1, z2, z4, z6 = np.zeros((1, 1)), np.zeros((2, 2)), np.zeros((4, 4)), np.zeros((6, 6))
    expected: List[np.ndarray] = []
    expected.append(fable.block_diag(i1, z2, z4, z6))  # R block
    expected.extend(
        [
            fable.block_diag(z1, i2, z4, z6),
            fable.block_diag(z1, j2, z4, z6),
        ]
    )
    for q in fable.quaternion_right_basis():
        expected.append(fable.block_diag(z1, z2, q, z6))
    for a in range(3):
        for b in range(3):
            eab = np.zeros((3, 3))
            eab[a, b] = 1.0
            expected.append(fable.block_diag(z1, z2, z4, np.kron(eab, i2)))
            expected.append(fable.block_diag(z1, z2, z4, np.kron(eab, j2)))

    expected_vec = np.column_stack([x.reshape(-1, order="F") for x in expected])
    q_expected, _ = np.linalg.qr(expected_vec)
    q_null, _ = np.linalg.qr(null_basis)
    r_ne = float(np.linalg.norm((np.eye(n * n) - q_expected @ q_expected.T) @ q_null, ord=2))
    r_en = float(np.linalg.norm((np.eye(n * n) - q_null @ q_null.T) @ q_expected, ord=2))
    max_comm = max(float(np.linalg.norm(x @ g - g @ x)) for x in expected for g in generators)

    return {
        "base_AF_witness": base,
        "extended_witness_group": "C14 x Q8 with one additional trivial real line",
        "real_representation_dimension": n,
        "constraint_matrix_shape": list(constraints.shape),
        "constraint_rank": rank,
        "computed_commutant_dimension": int(n * n - rank),
        "expected_algebra": "R + C + H + M3(C)",
        "expected_real_dimension": 25,
        "expected_basis_rank": int(np.linalg.matrix_rank(expected_vec, tol=1e-10)),
        "max_expected_basis_commutator_norm": max_comm,
        "span_residual_null_to_expected": r_ne,
        "span_residual_expected_to_null": r_en,
        "verdict": "PASS"
        if n * n - rank == 25
        and expected_vec.shape[1] == 25
        and max_comm < 1e-12
        and r_ne < 1e-10
        and r_en < 1e-10
        else "FAIL",
        "interpretation": (
            "A fixed trivial real sector adds an R summand to the commutant. "
            "This is the algebraic type exposed again by the source PDF's +1 dimension column."
        ),
    }


# ---------------------------------------------------------------------------
# III. Finite bimodule and order-one constraints
```

```python
# ---------------------------------------------------------------------------

# Ordered so gamma=+1 states come first:
#   particle-left, antiparticle-right | antiparticle-left, particle-right.
STATES: List[Tuple[int, int, int, int]] = []
for _a, _s in [(1, 1), (-1, -1), (-1, 1), (1, -1)]:
    for _w in (1, -1):
        for _c in range(4):
            STATES.append((_a, _s, _w, _c))
INDEX = {st: i for i, st in enumerate(STATES)}
GAMMA = np.diag([a * s for a, s, w, c in STATES]).astype(complex)
JPERM = np.zeros((32, 32), complex)
for i, (a, s, w, c) in enumerate(STATES):
    JPERM[INDEX[(-a, s, w, c)], i] = 1.0


def base_dirac_basis() -> np.ndarray:
    """Exact 272-real-dimensional basis for D*=D, {D,gamma}=0, DJ=JD."""
    out: List[np.ndarray] = []
    for p in range(16):
        for q in range(p, 16):
            for val in (1.0 + 0.0j, 1.0j):
                B = np.zeros((16, 16), complex)
                B[p, q] = val
                B[q, p] = val
                D = np.zeros((32, 32), complex)
                D[:16, 16:] = B
                D[16:, :16] = B.conj().T
                out.append(D)
    return np.asarray(out)


DBASE = base_dirac_basis()


def qmat(q: Sequence[float]) -> np.ndarray:
    q0, q1, q2, q3 = q
    return np.array(
        [[q0 + 1j * q1, q2 + 1j * q3], [-q2 + 1j * q3, q0 - 1j * q1]], complex
    )


def opposite(A: np.ndarray) -> np.ndarray:
    return JPERM @ A.T @ JPERM


def pi_extended(elem: Tuple[float, complex, np.ndarray, np.ndarray]) -> np.ndarray:
    """Linear unital representation of R + C + H + M3(C).

    The R summand acts on the four anti-lepton states. This is the unique simple
    linear repair of the source phrase "fixed identity action" that reproduces
    its displayed +1 dimensions and Dirac-space table.
    """
    r, lam, q, m = elem
    Q = qmat(q)
    A = np.zeros((32, 32), complex)
    for c in range(4):
        inds = [INDEX[(1, 1, 1, c)], INDEX[(1, 1, -1, c)]]
        A[np.ix_(inds, inds)] = Q
        A[INDEX[(1, -1, 1, c)], INDEX[(1, -1, 1, c)]] = lam
        A[INDEX[(1, -1, -1, c)], INDEX[(1, -1, -1, c)]] = lam.conjugate()
    for s in (1, -1):
        for w in (1, -1):
            inds = [INDEX[(-1, s, w, c)] for c in range(3)]
            A[np.ix_(inds, inds)] = m
            A[INDEX[(-1, s, w, 3)], INDEX[(-1, s, w, 3)]] = r
    return A


def pi_canonical(elem: Tuple[complex, np.ndarray, np.ndarray]) -> np.ndarray:
    """One natural linear unital completion of the PDF's written A_F action.

    Here the anti-lepton scalar is tied to lambda. This is not asserted to be
    the unique published NCG convention; it is a specification stress test of
    the text supplied in this conversation.
    """
    lam, q, m = elem
    Q = qmat(q)
    A = np.zeros((32, 32), complex)
    for c in range(4):
        inds = [INDEX[(1, 1, 1, c)], INDEX[(1, 1, -1, c)]]
        A[np.ix_(inds, inds)] = Q
        A[INDEX[(1, -1, 1, c)], INDEX[(1, -1, 1, c)]] = lam
        A[INDEX[(1, -1, -1, c)], INDEX[(1, -1, -1, c)]] = lam.conjugate()
    for s in (1, -1):
        for w in (1, -1):
            inds = [INDEX[(-1, s, w, c)] for c in range(3)]
            A[np.ix_(inds, inds)] = m
            A[INDEX[(-1, s, w, 3)], INDEX[(-1, s, w, 3)]] = lam
    return A


```

```python
def pi_affine_fixed_identity(elem: Tuple[complex, np.ndarray, np.ndarray]) -> np.ndarray:
    """Literal affine reading: anti-lepton block is I independently of elem."""
    lam, q, m = elem
    A = pi_canonical((lam, q, m))
    for s in (1, -1):
        for w in (1, -1):
            A[INDEX[(-1, s, w, 3)], INDEX[(-1, s, w, 3)]] = 1.0
    return A


def pi_ps(elem: Tuple[np.ndarray, np.ndarray, np.ndarray]) -> np.ndarray:
    qL, qR, m4 = elem
    QL, QR = qmat(qL), qmat(qR)
    A = np.zeros((32, 32), complex)
    for c in range(4):
        il = [INDEX[(1, 1, 1, c)], INDEX[(1, 1, -1, c)]]
        ir = [INDEX[(1, -1, 1, c)], INDEX[(1, -1, -1, c)]]
        A[np.ix_(il, il)] = QL
        A[np.ix_(ir, ir)] = QR
    for s in (1, -1):
        for w in (1, -1):
            inds = [INDEX[(-1, s, w, c)] for c in range(4)]
            A[np.ix_(inds, inds)] = m4
    return A


def extended_basis(flags: str) -> List[np.ndarray]:
    elems: List[Tuple[float, complex, np.ndarray, np.ndarray]] = [
        (1.0, 0j, np.zeros(4), np.zeros((3, 3), complex))
    ]
    if "C" in flags:
        for lam in (1.0 + 0j, 1.0j):
            elems.append((0.0, lam, np.zeros(4), np.zeros((3, 3), complex)))
    if "H" in flags:
        for j in range(4):
            q = np.zeros(4)
            q[j] = 1.0
            elems.append((0.0, 0j, q, np.zeros((3, 3), complex)))
    if "M" in flags:
        for a in range(3):
            for b in range(3):
                for z in (1.0 + 0j, 1.0j):
                    m = np.zeros((3, 3), complex)
                    m[a, b] = z
                    elems.append((0.0, 0j, np.zeros(4), m))
    return [pi_extended(e) for e in elems]


def canonical_basis() -> List[np.ndarray]:
    elems: List[Tuple[complex, np.ndarray, np.ndarray]] = []
    for lam in (1.0 + 0j, 1.0j):
        elems.append((lam, np.zeros(4), np.zeros((3, 3), complex)))
    for j in range(4):
        q = np.zeros(4)
        q[j] = 1.0
        elems.append((0j, q, np.zeros((3, 3), complex)))
    for a in range(3):
        for b in range(3):
            for z in (1.0 + 0j, 1.0j):
                m = np.zeros((3, 3), complex)
                m[a, b] = z
                elems.append((0j, np.zeros(4), m))
    return [pi_canonical(e) for e in elems]


def ps_basis() -> List[np.ndarray]:
    mats: List[np.ndarray] = []
    for side in (0, 1):
        for j in range(4):
            qL, qR = np.zeros(4), np.zeros(4)
            (qL if side == 0 else qR)[j] = 1.0
            mats.append(pi_ps((qL, qR, np.zeros((4, 4), complex))))
    for a in range(4):
        for b in range(4):
            for z in (1.0 + 0j, 1.0j):
                m = np.zeros((4, 4), complex)
                m[a, b] = z
                mats.append(pi_ps((np.zeros(4), np.zeros(4), m)))
    return mats


def shrink_basis(Dcur: np.ndarray, A: np.ndarray, B: np.ndarray, tol: float = 1e-9) -> np.ndarray:
    if len(Dcur) == 0:
        return Dcur
    C = Dcur @ A - A @ Dcur
    Y = C @ B - B @ C
    X = np.concatenate(
        [Y.reshape(len(Dcur), -1).real.T, Y.reshape(len(Dcur), -1).imag.T], axis=0
    )
    _, s, vh = la.svd(X, full_matrices=False, lapack_driver="gesdd", check_finite=False)
    threshold = max(1e-10, tol * (s[0] if len(s) and s[0] > 0 else 1.0))
    rank = int(np.sum(s > threshold))
```

```
    if rank == 0:
        return Dcur
    if rank >= len(Dcur):
        return np.zeros((0, 32, 32), complex)
    N = vh[rank:].T
    return np.einsum("ji,jab->iab", N, Dcur, optimize=True)


def numerical_order_one_space(mats: Sequence[np.ndarray], seed: int = 10) -> Tuple[np.ndarray, float]:
    ops = [opposite(A) for A in mats]
    Dcur = DBASE.copy()
    norms = np.linalg.norm(Dcur.reshape(len(Dcur), -1), axis=1)
    Dcur = Dcur / norms[:, None, None]
    rng = np.random.default_rng(seed)
    for _ in range(8):
        A = sum(float(rng.normal()) * x for x in mats)
        B = sum(float(rng.normal()) * x for x in ops)
        Dcur = shrink_basis(Dcur, A, B)
    for A in mats:
        for B in ops:
            Dcur = shrink_basis(Dcur, A, B)
    max_resid = 0.0
    for A in mats:
        for B in ops:
            C = Dcur @ A - A @ Dcur
            max_resid = max(max_resid, float(np.linalg.norm(C @ B - B @ C)))
    return Dcur, max_resid


def physical_yukawa_basis() -> np.ndarray:
    out: List[np.ndarray] = []
    # In the gamma ordering: rows 0..7 particle-L; columns 8..15 particle-R.
    for typ in ("q", "l"):
        colors: Iterable[int] = range(3) if typ == "q" else [3]
        for wl_i in range(2):
            for wr_i in range(2):
                for imag in (False, True):
                    B = np.zeros((16, 16), complex)
                    val = 1j if imag else 1.0
                    for c in colors:
                        pl = wl_i * 4 + c
                        pr = 8 + wr_i * 4 + c
                        B[pl, pr] = val
                        B[pr, pl] = val
                    D = np.zeros((32, 32), complex)
                    D[:16, 16:] = B
                    D[16:, :16] = B.conj().T
                    out.append(D)
    return np.asarray(out)


def paired_ps_yukawa_basis() -> np.ndarray:
    y = physical_yukawa_basis()
    return y[:8] + y[8:]


def max_order_one_residual(Ys: np.ndarray, mats: Sequence[np.ndarray]) -> float:
    ops = [opposite(A) for A in mats]
    mx = 0.0
    for A in mats:
        for B in ops:
            C = Ys @ A - A @ Ys
            Z = C @ B - B @ C
            mx = max(mx, float(np.max(np.abs(Z))))
    return mx


def real_span_rank(Ys: np.ndarray, tol: float = 1e-12) -> int:
    V = np.concatenate(
        [Ys.reshape(len(Ys), -1).real, Ys.reshape(len(Ys), -1).imag], axis=1
    )
    return int(np.linalg.matrix_rank(V, tol=tol))


def constraint_matrix(Dbase: np.ndarray, A: np.ndarray, B: np.ndarray) -> np.ndarray:
    C = Dbase @ A - A @ Dbase
    Y = C @ B - B @ C
    F = Y.reshape(len(Dbase), -1)
    X = np.concatenate([F.real.T, F.imag.T], axis=0)
    rounded = np.rint(X)
    if float(np.max(np.abs(X - rounded))) > 1e-10:
        raise RuntimeError("Expected integer constraint matrix")
    return rounded.astype(np.int64)


def rank_mod(M: np.ndarray, p: int) -> int:
    A = np.mod(M, p).astype(np.int64, copy=True)
    m, n = A.shape
    r = 0
    for c in range(n):
        nz = np.flatnonzero(A[r:, c])
        if nz.size == 0:
```

```
            continue
        i = r + int(nz[0])
        if i != r:
            A[[r, i]] = A[[i, r]]
        inv = pow(int(A[r, c]), -1, p)
        A[r, c:] = (A[r, c:] * inv) % p
        if r + 1 < m:
            factors = A[r + 1 :, c].copy()
            mask = factors != 0
            if np.any(mask):
                rows = np.where(mask)[0] + r + 1
                A[rows, c:] = (A[rows, c:] - factors[mask, None] * A[r, c:]) % p
        r += 1
        if r == m or r == n:
            break
    return r


def random_extended_elem(rng: np.random.Generator):
    r = int(rng.integers(-2, 3))
    lam = complex(int(rng.integers(-2, 3)), int(rng.integers(-2, 3)))
    q = rng.integers(-2, 3, size=4).astype(float)
    m = rng.integers(-2, 3, size=(3, 3)) + 1j * rng.integers(-2, 3, size=(3, 3))
    return r, lam, q, m


def random_ps_elem(rng: np.random.Generator):
    qL = rng.integers(-2, 3, size=4).astype(float)
    qR = rng.integers(-2, 3, size=4).astype(float)
    m = rng.integers(-2, 3, size=(4, 4)) + 1j * rng.integers(-2, 3, size=(4, 4))
    return qL, qR, m


def exact_rank_sandwich_receipt() -> Dict[str, object]:
    primes = [1009, 1013, 10007, 10009]
    rng = np.random.default_rng(20260803)

    # Two generic constraints already expose rank 256 for the extended model.
    X_ext_parts = []
    for _ in range(2):
        A = pi_extended(random_extended_elem(rng))
        B = opposite(pi_extended(random_extended_elem(rng)))
        X_ext_parts.append(constraint_matrix(DBASE, A, B))
    X_ext = np.concatenate(X_ext_parts, axis=0)
    ext_ranks = {str(p): rank_mod(X_ext, p) for p in primes}

    # One generic PS constraint exposes rank 264.
    Aps = pi_ps(random_ps_elem(rng))
    Bps = opposite(pi_ps(random_ps_elem(rng)))
    X_ps = constraint_matrix(DBASE, Aps, Bps)
    ps_ranks = {str(p): rank_mod(X_ps, p) for p in primes}

    Y16 = physical_yukawa_basis()
    Y8 = paired_ps_yukawa_basis()
    ext_mats = extended_basis("CHM")
    ps_mats = ps_basis()
    y16_res = max_order_one_residual(Y16, ext_mats)
    y8_res = max_order_one_residual(Y8, ps_mats)
    y16_rank = real_span_rank(Y16)
    y8_rank = real_span_rank(Y8)

    ext_exact = all(r == 256 for r in ext_ranks.values()) and y16_rank == 16 and y16_res == 0.0
    ps_exact = all(r == 264 for r in ps_ranks.values()) and y8_rank == 8 and y8_res == 0.0

    return {
        "method": (
            "rank lower bound from a nonzero modular minor; nullity upper bound from that rank; "
            "matching exact explicit null vectors give the reverse inequality"
        ),
        "extended_model": {
            "subset_constraint_shape": list(X_ext.shape),
            "modular_ranks": ext_ranks,
            "rank_lower_bound_over_Q": min(ext_ranks.values()),
            "explicit_null_vectors": y16_rank,
            "full_basis_max_exact_entry_residual": y16_res,
            "exact_nullity": 16 if ext_exact else None,
            "verdict": "EXACT" if ext_exact else "FAIL",
        },
        "pati_salam": {
            "subset_constraint_shape": list(X_ps.shape),
            "modular_ranks": ps_ranks,
            "rank_lower_bound_over_Q": min(ps_ranks.values()),
            "explicit_null_vectors": y8_rank,
            "full_basis_max_exact_entry_residual": y8_res,
            "exact_nullity": 8 if ps_exact else None,
            "verdict": "EXACT" if ps_exact else "FAIL",
        },
    }


def bimodule_receipt() -> Dict[str, object]:
    t0 = time.time()
```

```
    base_checks = {
        "basis_shape": list(DBASE.shape),
        "analytic_dimension": 16 * 17,
        "max_hermiticity_residual": max(float(np.linalg.norm(D - D.conj().T)) for D in DBASE),
        "max_anticommute_gamma_residual": max(float(np.linalg.norm(D @ GAMMA + GAMMA @ D)) for D in DBASE),
        "max_commute_J_residual": max(
            float(np.linalg.norm(D @ JPERM - JPERM @ D.conjugate())) for D in DBASE
        ),
        "J2_residual": float(np.linalg.norm(JPERM @ JPERM - np.eye(32))),
        "Jgamma_anticommutator_residual": float(np.linalg.norm(JPERM @ GAMMA + GAMMA @ JPERM)),
    }

    # Source-aligned displayed table. The first row is the global unit only;
    # all nontrivial rows reproduce only after adjoining the anti-lepton R block.
    row_specs = [
        ("C", "C", 3, 146),
        ("H", "H", 5, 146),
        ("C+H", "CH", 7, 80),
        ("M3(C)", "M", 19, 128),
        ("C+M3(C)", "CM", 21, 16),
        ("H+M3(C)", "HM", 23, 16),
        ("C+H+M3(C)", "CHM", 25, 16),
    ]
    source_rows: List[Dict[str, object]] = [
        {
            "label": "none (global unit only)",
            "represented_real_dimension_in_pdf": 1,
            "computed_D_dimension": 272,
            "pdf_reported_D_dimension": 272,
            "residual": 0.0,
            "model": "global identity imposes no order-one constraint",
        }
    ]
    for label, flags, pdf_alg_dim, pdf_D_dim in row_specs:
        mats = extended_basis(flags)
        Dcur, resid = numerical_order_one_space(mats)
        source_rows.append(
            {
                "label": label,
                "flags": flags,
                "represented_real_dimension_in_pdf": pdf_alg_dim,
                "extended_algebra_dimension": len(mats),
                "computed_D_dimension": int(len(Dcur)),
                "pdf_reported_D_dimension": pdf_D_dim,
                "matches_pdf": bool(len(Dcur) == pdf_D_dim and len(mats) == pdf_alg_dim),
                "full_basis_residual": resid,
                "linear_closure": "R + " + label,
            }
        )

    # Standard A_F real dimension and the simplest linear unital completion.
    canonical_mats = canonical_basis()
    Dcan, canonical_resid = numerical_order_one_space(canonical_mats)

    # Literal fixed-identity phrase is affine, not linear.
    z = (0j, np.zeros(4), np.zeros((3, 3), complex))
    a = (1 + 2j, np.array([1.0, -1.0, 2.0, 0.0]), np.eye(3, dtype=complex))
    b = (-2 + 1j, np.array([0.0, 1.0, 0.0, -1.0]), 2j * np.eye(3, dtype=complex))
    affine_zero = pi_affine_fixed_identity(z)
    affine_add = pi_affine_fixed_identity(
        (a[0] + b[0], a[1] + b[1], a[2] + b[2])
    ) - pi_affine_fixed_identity(a) - pi_affine_fixed_identity(b)

    exact = exact_rank_sandwich_receipt()

    # PS numerical cross-check beyond exact sandwich.
    Dps, ps_resid = numerical_order_one_space(ps_basis())

    source_match = all(bool(row.get("matches_pdf", True)) for row in source_rows)
    return {
        "status": "AUDIT_WITH_EXACT_RANK_CERTIFICATES",
        "runtime_seconds": time.time() - t0,
        "base_space": base_checks,
        "source_pdf_table_reconstruction": source_rows,
        "all_displayed_rows_reproduced_by_extended_model": source_match,
        "hidden_unit_diagnosis": {
            "standard_AF_real_dimension": 2 + 4 + 18,
            "pdf_displayed_full_dimension": 25,
            "difference": 1,
            "linear_closure_that_reproduces_table": "R + C + H + M3(C)",
            "extended_real_dimension": 1 + 2 + 4 + 18,
            "interpretation": (
                "The anti-lepton 'fixed identity' behaves as an independent real projector. "
                "Promoting it to a scalar makes the action linear and unital, but changes the algebra."
            ),
        },
        "literal_affine_reading": {
            "pi_of_zero_frobenius_norm": float(np.linalg.norm(affine_zero)),
            "additivity_failure_frobenius_norm": float(np.linalg.norm(affine_add)),
            "verdict": "NOT_A_LINEAR_REPRESENTATION",
        },
        "canonical_text_completion": {
```

```
            "model": "anti-lepton scalar tied to lambda",
            "algebra_real_dimension": 24,
            "computed_D_dimension": int(len(Dcan)),
            "full_basis_residual": canonical_resid,
            "status": "COMPUTED_SPECIFICATION_STRESS_TEST",
            "boundary": (
                "This is one natural completion of the supplied text, not a claim about all published NCG conventions."
            ),
        },
        "source_aligned_extended_model": {
            "algebra": "R + C + H + M3(C)",
            "real_dimension": 25,
            "computed_D_dimension": 16,
            "status": exact["extended_model"]["verdict"],
        },
        "pati_salam": {
            "algebra": "H_L + H_R + M4(C)",
            "computed_D_dimension": int(len(Dps)),
            "full_basis_residual": ps_resid,
            "status": exact["pati_salam"]["verdict"],
        },
        "exact_rank_sandwich": exact,
        "step4_boundary": {
            "relative_step4_on_standard_AF": "NOT_CLOSED_BY_THIS_SUPPLIED_SPECIFICATION",
            "reason": (
                "The reported 16-dimensional plateau is exactly certified for the extended R+AF action, "
                "while a natural linear unital AF completion gives 32. The representation must be fixed before selection."
            ),
            "global_step4": "OPEN",
        },
    }


# ---------------------------------------------------------------------------
# IV. Summary figures and certificate
# ---------------------------------------------------------------------------

def make_summary_figures(receipt: Dict[str, object]) -> None:
    import matplotlib.pyplot as plt

    # Tower flow diagram.
    fig, ax = plt.subplots(figsize=(12, 4.8))
    ax.axis("off")
    labels = [
        "Mosaic\nfinite image",
        "C14 / D14\nplanar symmetry",
        "Schur lift\nC, H, C^3",
        "commutant\nA_F (24)",
        "fixed real line\nR + A_F (25)",
        "order-one\n16 vs PS 8",
        "Step 4\nspecification fork",
    ]
    xs = np.linspace(0.06, 0.94, len(labels))
    y = 0.55
    for i, (x, lab) in enumerate(zip(xs, labels)):
        ax.text(
            x,
            y,
            lab,
            ha="center",
            va="center",
            fontsize=10,
            bbox=dict(boxstyle="round,pad=0.45", facecolor="white", edgecolor="black"),
            transform=ax.transAxes,
        )
        if i < len(labels) - 1:
            ax.annotate(
                "",
                xy=(xs[i + 1] - 0.055, y),
                xytext=(x + 0.055, y),
                arrowprops=dict(arrowstyle="->", lw=1.5),
                xycoords=ax.transAxes,
            )
    ax.text(
        0.5,
        0.12,
        "Exact where algebraic; computed where image- or rank-numerical; global Standard-Model origin remains open.",
        ha="center",
        va="center",
        fontsize=10,
        transform=ax.transAxes,
    )
    fig.tight_layout()
    fig.savefig(FIG / "tower_flow.png", dpi=200)
    plt.close(fig)

    # Source table comparison.
    rows = receipt["bimodule"]["source_pdf_table_reconstruction"]
    labels = [r["label"] for r in rows]
    reported = [r["pdf_reported_D_dimension"] for r in rows]
    computed = [r["computed_D_dimension"] for r in rows]
    x = np.arange(len(labels))
```

```
    fig, ax = plt.subplots(figsize=(11.5, 5.4))
    width = 0.38
    ax.bar(x - width / 2, reported, width, label="PDF reported")
    ax.bar(x + width / 2, computed, width, label="reconstructed R+... model")
    ax.set_xticks(x, labels, rotation=28, ha="right")
    ax.set_ylabel("real dimension of admissible Dirac space")
    ax.set_title("Source table reproduced only by the unit-completed extended representation")
    ax.legend()
    fig.tight_layout()
    fig.savefig(FIG / "step4_table_reconstruction.png", dpi=200)
    plt.close(fig)

    # Branch comparison.
    fig, ax = plt.subplots(figsize=(7.5, 4.8))
    names = ["standard A_F\ncanonical completion", "R + A_F\nsource-aligned", "Pati-Salam"]
    dims = [
        receipt["bimodule"]["canonical_text_completion"]["computed_D_dimension"],
        receipt["bimodule"]["source_aligned_extended_model"]["computed_D_dimension"],
        receipt["bimodule"]["pati_salam"]["computed_D_dimension"],
    ]
    ax.bar(names, dims)
    ax.set_ylabel("dim_R D")
    ax.set_title("The representation convention changes the order-one result")
    for i, v in enumerate(dims):
        ax.text(i, v + 0.7, str(v), ha="center", fontweight="bold")
    ax.set_ylim(0, max(dims) * 1.18)
    fig.tight_layout()
    fig.savefig(FIG / "representation_fork.png", dpi=200)
    plt.close(fig)


def main() -> None:
    started = time.time()
    fable = run_fable_image_audit()
    light = light_matrix_receipt(depth=1000)
    commutants = extended_commutant_receipt()
    bimodule = bimodule_receipt()

    receipt: Dict[str, object] = {
        "artifact": "SOL FABLE LATEX TOWER v1.0",
        "generated_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "status_grammar": {
            "EXACT": "symbolic/integer identity or exact rank sandwich",
            "COMPUTED": "finite numerical calculation with residual",
            "CONDITIONAL": "exact after explicit representation assumptions",
            "OPEN": "not selected or derived",
        },
        "inputs": {
            "genesis_alpha_html_sha256": sha256(SRC / "shell__genesis_alpha_v0_1(1).html"),
            "light_matrix_receipt_sha256": sha256(SRC / "light_matrix_receipt(1).py"),
            "strange_idea_pdf_sha256": sha256(SRC / "strange_idea_continued.pdf"),
            "mosaic_sha256": sha256(ROOT / "mosaic_rectified.png"),
        },
        "fable": fable,
        "light_matrix": light,
        "commutants": commutants,
        "bimodule": bimodule,
        "final_verdict": {
            "mosaic_directly_encodes_standard_model": False,
            "light_matrix_exact": True,
            "genesis_shifted_power_is_literal_GC_refinement": False,
            "conditional_commutant_reconstructs_standard_AF": True,
            "fixed_real_line_reconstructs_extended_R_plus_AF": True,
            "source_pdf_16_plateau_reproduced": True,
            "source_pdf_16_plateau_algebra": "R + C + H + M3(C)",
            "standard_AF_step4_closed": False,
            "global_step4_closed": False,
        },
        "runtime_seconds": time.time() - started,
    }
    make_summary_figures(receipt)
    json_dump(REC / "sol_fable_tower_v1_receipt.json", receipt)

    lines = [
        "SOL FABLE LATEX TOWER v1.0 - compact result ledger",
        "",
        f"runtime_seconds: {receipt['runtime_seconds']:.3f}",
        f"mosaic_inner_mode: {fable['image']['strongest_inner_mode']}",
        f"mosaic_outer_mode: {fable['image']['strongest_outer_mode']}",
        f"commutant_AF_dimension: {commutants['base_AF_witness']['computed_commutant_dimension']}",
        f"commutant_R_plus_AF_dimension: {commutants['computed_commutant_dimension']}",
        f"base_D_dimension: {bimodule['base_space']['analytic_dimension']}",
        f"source_aligned_D_dimension: {bimodule['source_aligned_extended_model']['computed_D_dimension']}",
        f"canonical_AF_completion_D_dimension: {bimodule['canonical_text_completion']['computed_D_dimension']}",
        f"pati_salam_D_dimension: {bimodule['pati_salam']['computed_D_dimension']}",
        f"source_table_matches_R_plus_model: {bimodule['all_displayed_rows_reproduced_by_extended_model']}",
        f"genesis_counterexample_T: {light['genesis_alpha_counterexample']['actual_next_T']} != {light['genesis_alpha_counterexample']['claimed_power_T']}",
        "standard_AF_step4: OPEN / representation specification unresolved",
        "global_step4: OPEN",
    ]
    (REC / "sol_fable_tower_v1_results.txt").write_text("\n".join(lines) + "\n", encoding="utf-8")
```

```
    print("\n".join(lines))


if __name__ == "__main__":
    main()
```

E Code block E: complete Corinth-image and Schur-witness audit

```
#!/usr/bin/env python3
"""FABLE v0.2: image/symmetry and algebraic-closure audit.

This script performs four independent checks:
  1. angular Fourier analysis of the inner volute annulus;
  2. angular Fourier analysis of the outer triangular/lens field;
  3. exact Wedderburn dimension checks for C_14 and D_14 group algebras;
  4. local Step-4 selector/no-go checks using the dimensions reported in
     ``A Strange Idea, Followed Further''.

The script does not claim that the mosaic is an exact logarithmic-spiral
construction. It tests the finite image that was supplied.
"""
from __future__ import annotations

import json
import math
from pathlib import Path
from typing import Dict, List, Tuple

import cv2
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

ROOT = Path(__file__).resolve().parent
IMAGE = ROOT / "mosaic_rectified.png"
OUT_JSON = ROOT / "fable_v0_2_receipt.json"
OUT_TXT = ROOT / "fable_v0_2_results.txt"
OUT_SPECTRUM = ROOT / "fable_mode_spectrum.png"
OUT_POLAR = ROOT / "fable_polar_strip.png"

# The supplied image was already rectified to 900 x 900. The medallion centre
# is fixed from the central circular frame, not optimized over Fourier mode.
CENTER = (451.0, 456.0)  # (x, y), pixels
INNER_ANNULUS = (85.0, 155.0)
OUTER_ANNULUS = (180.0, 380.0)
N_ANGLE = 4096
N_RADIAL = 160
MAX_MODE = 40

# Small perturbation grid used to test that the dominant modes are not artifacts
# of a single centre/annulus choice. 25 centres x 7 annuli = 175 trials.
CENTER_OFFSETS = (-6.0, -3.0, 0.0, 3.0, 6.0)
INNER_ANNULI_SWEEP = ((75.0,145.0),(80.0,150.0),(85.0,155.0),
                      (90.0,160.0),(95.0,165.0),(80.0,160.0),(75.0,165.0))
OUTER_ANNULI_SWEEP = ((160.0,340.0),(170.0,350.0),(180.0,360.0),
                      (180.0,380.0),(190.0,370.0),(200.0,380.0),(170.0,390.0))


def load_features(path: Path) -> Dict[str, np.ndarray]:
    rgb = np.asarray(Image.open(path).convert("RGB"), dtype=np.float64) / 255.0
    gray = cv2.cvtColor((rgb * 255).astype(np.uint8), cv2.COLOR_RGB2GRAY).astype(np.float64) / 255.0
    gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    edge = np.hypot(gx, gy)
    redness = rgb[:, :, 0] - 0.5 * (rgb[:, :, 1] + rgb[:, :, 2])
    return {"rgb": rgb, "gray": gray, "edge": edge, "redness": redness}


def polar_samples(
    feature: np.ndarray,
    center: Tuple[float, float],
    annulus: Tuple[float, float],
    n_radial: int = N_RADIAL,
    n_angle: int = N_ANGLE,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    cx, cy = center
    r_min, r_max = annulus
    theta = np.linspace(0.0, 2.0 * math.pi, n_angle, endpoint=False)
    radius = np.linspace(r_min, r_max, n_radial)
    x_map = (cx + radius[:, None] * np.cos(theta)[None, :]).astype(np.float32)
    y_map = (cy + radius[:, None] * np.sin(theta)[None, :]).astype(np.float32)
    values = cv2.remap(
        feature.astype(np.float32),
        x_map,
        y_map,
        interpolation=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_REFLECT,
    )
```

```
    return radius, theta, values


def angular_power(values: np.ndarray, max_mode: int = MAX_MODE) -> np.ndarray:
    centered = values - values.mean(axis=1, keepdims=True)
    spectrum = np.fft.rfft(centered, axis=1)
    power = np.mean(np.abs(spectrum[:, : max_mode + 1]) ** 2, axis=0)
    return power


def ranked_modes(power: np.ndarray, top: int = 10) -> List[Dict[str, float]]:
    total = float(power[1:].sum())
    order = np.argsort(power[1:])[::-1] + 1
    return [
        {
            "mode": int(m),
            "power": float(power[m]),
            "fraction_modes_1_to_max": float(power[m] / total),
        }
        for m in order[:top]
    ]


def write_polar_strip(rgb: np.ndarray) -> None:
    # A direct polar strip is easier to audit visually than the Fourier table.
    bgr = cv2.cvtColor((rgb * 255).astype(np.uint8), cv2.COLOR_RGB2BGR)
    max_radius = 200
    polar = cv2.warpPolar(
        bgr,
        (max_radius, 1440),
        CENTER,
        max_radius,
        cv2.WARP_POLAR_LINEAR + cv2.WARP_FILL_OUTLIERS,
    )
    strip = polar[:, int(INNER_ANNULUS[0]) : int(INNER_ANNULUS[1]), :]
    strip = np.transpose(strip, (1, 0, 2))
    cv2.imwrite(str(OUT_POLAR), strip)


def plot_mode_spectrum(inner: np.ndarray, outer: np.ndarray) -> None:
    modes = np.arange(1, MAX_MODE + 1)
    inner_n = inner[1:] / inner[1:].sum()
    outer_n = outer[1:] / outer[1:].sum()

    fig, ax = plt.subplots(figsize=(10, 4.8))
    width = 0.42
    ax.bar(modes - width / 2, inner_n, width=width, label="inner volute annulus (redness)")
    ax.bar(modes + width / 2, outer_n, width=width, label="outer field (edge magnitude)")
    ax.set_xlabel("angular Fourier mode m")
    ax.set_ylabel("fraction of power over modes 1..40")
    ax.set_xlim(0.5, MAX_MODE + 0.5)
    ax.set_title("FABLE v0.2 - angular mode audit of the supplied Corinth mosaic image")
    ax.legend()
    fig.tight_layout()
    fig.savefig(OUT_SPECTRUM, dpi=180)
    plt.close(fig)



def robustness_sweep(
    feature: np.ndarray,
    target_mode: int,
    annuli: Tuple[Tuple[float, float], ...],
    n_radial: int = 100,
    n_angle: int = 2048,
) -> Dict[str, object]:
    """Perturb centre and annulus and record the target mode's stability."""
    tops: List[int] = []
    ranks: List[int] = []
    fractions: List[float] = []
    for dx in CENTER_OFFSETS:
        for dy in CENTER_OFFSETS:
            centre = (CENTER[0] + dx, CENTER[1] + dy)
            for annulus in annuli:
                _, _, values = polar_samples(
                    feature, centre, annulus, n_radial=n_radial, n_angle=n_angle
                )
                p = angular_power(values)
                order = np.argsort(p[1:])[::-1] + 1
                tops.append(int(order[0]))
                ranks.append(int(np.where(order == target_mode)[0][0] + 1))
                fractions.append(float(p[target_mode] / p[1:].sum()))
    trials = len(tops)
    counts = {str(m): int(tops.count(m)) for m in sorted(set(tops))}
    return {
        "trials": trials,
        "target_mode": target_mode,
        "target_is_top_count": int(sum(m == target_mode for m in tops)),
        "target_is_top_fraction": float(sum(m == target_mode for m in tops) / trials),
        "median_target_rank": float(np.median(ranks)),
        "median_target_power_fraction": float(np.median(fractions)),
        "top_mode_counts": counts,
```

```
    }


def rotation(theta: float) -> np.ndarray:
    return np.array(
        [[math.cos(theta), -math.sin(theta)],
         [math.sin(theta),  math.cos(theta)]],
        dtype=np.float64,
    )


def block_diag(*blocks: np.ndarray) -> np.ndarray:
    size = sum(block.shape[0] for block in blocks)
    out = np.zeros((size, size), dtype=np.float64)
    cursor = 0
    for block in blocks:
        n = block.shape[0]
        out[cursor:cursor+n, cursor:cursor+n] = block
        cursor += n
    return out


def quaternion_left_generators() -> Tuple[np.ndarray, np.ndarray]:
    # Basis (1,i,j,k). These matrices implement left multiplication by i and j.
    li = np.array([
        [0,-1, 0, 0],
        [1, 0, 0, 0],
        [0, 0, 0,-1],
        [0, 0, 1, 0],
    ], dtype=np.float64)
    lj = np.array([
        [0, 0,-1, 0],
        [0, 0, 0, 1],
        [1, 0, 0, 0],
        [0,-1, 0, 0],
    ], dtype=np.float64)
    return li, lj


def quaternion_right_basis() -> List[np.ndarray]:
    ident = np.eye(4)
    ri = np.array([
        [0,-1, 0, 0],
        [1, 0, 0, 0],
        [0, 0, 0, 1],
        [0, 0,-1, 0],
    ], dtype=np.float64)
    rj = np.array([
        [0, 0,-1, 0],
        [0, 0, 0,-1],
        [1, 0, 0, 0],
        [0, 1, 0, 0],
    ], dtype=np.float64)
    rk = ri @ rj
    return [ident, ri, rj, rk]


def explicit_schur_witness() -> Dict[str, object]:
    """Construct a C14 x Q8 representation whose commutant is A_F.

    This is a witness for the conditional Schur theorem, not a derivation from
    the mosaic. The two C14 frequencies are inequivalent complex-type real
    irreps; the Q8 block is quaternionic; the second complex irrep occurs with
    multiplicity three.
    """
    r1 = rotation(2.0 * math.pi / 14.0)
    r3 = rotation(6.0 * math.pi / 14.0)
    li, lj = quaternion_left_generators()
    i2, i4, i6 = np.eye(2), np.eye(4), np.eye(6)
    color_rot = np.kron(np.eye(3), r3)

    generators = [
        block_diag(r1, i4, color_rot),
        block_diag(i2, li, i6),
        block_diag(i2, lj, i6),
    ]
    n = generators[0].shape[0]
    eye = np.eye(n)
    constraints = np.vstack([
        np.kron(g.T, eye) - np.kron(eye, g) for g in generators
    ])
    _, singular, vh = np.linalg.svd(constraints, full_matrices=True)
    tol = 1e-10 * singular[0]
    rank = int(np.sum(singular > tol))
    null_basis = vh.T[:, rank:]

    j2 = np.array([[0.0,-1.0],[1.0,0.0]])
    expected: List[np.ndarray] = []
    # C block.
    expected.extend([block_diag(i2, np.zeros((4,4)), np.zeros((6,6))),
                     block_diag(j2, np.zeros((4,4)), np.zeros((6,6)))])
    # H block: right multiplication commutes with left Q8 action.
```

```
    for q in quaternion_right_basis():
        expected.append(block_diag(np.zeros((2,2)), q, np.zeros((6,6))))
    # M_3(C) block: E_ab tensor {I,J}.
    for a in range(3):
        for b in range(3):
            eab = np.zeros((3,3)); eab[a,b] = 1.0
            expected.append(block_diag(np.zeros((2,2)), np.zeros((4,4)), np.kron(eab, i2)))
            expected.append(block_diag(np.zeros((2,2)), np.zeros((4,4)), np.kron(eab, j2)))

    expected_vec = np.column_stack([x.reshape(-1, order="F") for x in expected])
    q_expected, _ = np.linalg.qr(expected_vec)
    q_null, _ = np.linalg.qr(null_basis)
    span_residual_null_to_expected = float(
        np.linalg.norm((np.eye(n*n) - q_expected @ q_expected.T) @ q_null, ord=2)
    )
    span_residual_expected_to_null = float(
        np.linalg.norm((np.eye(n*n) - q_null @ q_null.T) @ q_expected, ord=2)
    )
    max_commutator = 0.0
    for x in expected:
        for g in generators:
            max_commutator = max(max_commutator, float(np.linalg.norm(x @ g - g @ x)))

    return {
        "witness_group": "C14 x Q8",
        "real_representation_dimension": n,
        "sector_data": ["(C, multiplicity 1)", "(H, multiplicity 1)", "(C, multiplicity 3)"],
        "constraint_matrix_shape": list(constraints.shape),
        "constraint_rank": rank,
        "computed_commutant_dimension": int(n*n - rank),
        "expected_AF_real_dimension": 24,
        "expected_basis_rank": int(np.linalg.matrix_rank(expected_vec, tol=1e-10)),
        "max_expected_basis_commutator_norm": max_commutator,
        "span_residual_null_to_expected": span_residual_null_to_expected,
        "span_residual_expected_to_null": span_residual_expected_to_null,
        "verdict": "PASS" if (n*n-rank == 24 and expected_vec.shape[1] == 24 and max_commutator < 1e-12 and
     span_residual_null_to_expected < 1e-10 and span_residual_expected_to_null < 1e-10) else "FAIL",
        "boundary": "The Q8 lift and multiplicity-three decoration are supplied assumptions; the image does not force them.",
    }


def q8_real_group_algebra() -> Dict[str, int]:
    # R[Q8] ~= R^4 + H; dimensions 4 + 4 = 8.
    return {"R_blocks": 4, "H_blocks": 1, "real_dimension": 8}


def cyclic_real_group_algebra_even(n: int) -> Dict[str, int]:
    if n % 2 != 0:
        raise ValueError("This closed form is for even n.")
    # R[C_n] ~= R^2 + C^((n-2)/2)
    r_blocks = 2
    c_blocks = (n - 2) // 2
    real_dimension = r_blocks + 2 * c_blocks
    assert real_dimension == n
    return {"R_blocks": r_blocks, "C_blocks": c_blocks, "real_dimension": real_dimension}


def dihedral_real_group_algebra_even(n: int) -> Dict[str, int]:
    if n % 2 != 0:
        raise ValueError("This closed form is for even n.")
    # D_n has order 2n. For even n:
    # R[D_n] ~= R^4 + M_2(R)^((n/2)-1).
    r_blocks = 4
    m2r_blocks = n // 2 - 1
    real_dimension = r_blocks + 4 * m2r_blocks
    assert real_dimension == 2 * n
    return {"R_blocks": r_blocks, "M2R_blocks": m2r_blocks, "real_dimension": real_dimension}


def standard_model_algebra_signature() -> Dict[str, int]:
    # A_F = C + H + M_3(C), standard real dimensions 2 + 4 + 18 = 24.
    return {
        "C_blocks": 1,
        "H_blocks": 1,
        "M3C_blocks": 1,
        "real_dimension": 24,
    }


def schur_lift_signature(m_complex_1: int, m_quaternionic: int, m_complex_3: int) -> Dict[str, int]:
    # Commutant of three inequivalent isotypic sectors with Schur division
    # algebras C, H, C and multiplicities m_complex_1, m_quaternionic,
    # m_complex_3 respectively.
    return {
        "M_C_size": m_complex_1,
        "M_H_size": m_quaternionic,
        "M_C_color_size": m_complex_3,
        "matches_AF": bool((m_complex_1, m_quaternionic, m_complex_3) == (1, 1, 3)),
    }


def local_step4_table() -> List[Dict[str, object]]:
```

```
    # Values are the results reported in the supplied paper. The boolean columns
    # are the two representation discriminators identified there.
    return [
        {
            "algebra": "C + M3(C)",
            "dirac_dim": 16,
            "weak_SU2": False,
            "uR_dR_character_split": True,
            "quark_lepton_yukawas_independent": True,
        },
        {
            "algebra": "H + M3(C)",
            "dirac_dim": 16,
            "weak_SU2": True,
            "uR_dR_character_split": False,
            "quark_lepton_yukawas_independent": True,
        },
        {
            "algebra": "C + H + M3(C)",
            "dirac_dim": 16,
            "weak_SU2": True,
            "uR_dR_character_split": True,
            "quark_lepton_yukawas_independent": True,
        },
        {
            "algebra": "H_L + H_R + M4(C)",
            "dirac_dim": 8,
            "weak_SU2": True,
            "uR_dR_character_split": False,
            "quark_lepton_yukawas_independent": False,
        },
    ]


def relative_defect(row: Dict[str, object]) -> Tuple[int, int, int, int]:
    return (
        max(0, 16 - int(row["dirac_dim"])),
        0 if bool(row["weak_SU2"]) else 1,
        0 if bool(row["uR_dR_character_split"]) else 1,
        0 if bool(row["quark_lepton_yukawas_independent"]) else 1,
    )


def main() -> None:
    features = load_features(IMAGE)

    _, _, inner_values = polar_samples(features["redness"], CENTER, INNER_ANNULUS)
    _, _, outer_values = polar_samples(features["edge"], CENTER, OUTER_ANNULUS)
    inner_power = angular_power(inner_values)
    outer_power = angular_power(outer_values)

    inner_modes = ranked_modes(inner_power)
    outer_modes = ranked_modes(outer_power)
    inner_robust_red = robustness_sweep(features["redness"], 14, INNER_ANNULI_SWEEP)
    inner_robust_edge = robustness_sweep(features["edge"], 14, INNER_ANNULI_SWEEP)
    outer_robust_red = robustness_sweep(features["redness"], 8, OUTER_ANNULI_SWEEP, n_radial=140)
    outer_robust_edge = robustness_sweep(features["edge"], 8, OUTER_ANNULI_SWEEP, n_radial=140)
    write_polar_strip(features["rgb"])
    plot_mode_spectrum(inner_power, outer_power)

    observed_inner_mode = int(inner_modes[0]["mode"])
    observed_outer_mode = int(outer_modes[0]["mode"])

    c14 = cyclic_real_group_algebra_even(observed_inner_mode)
    d14 = dihedral_real_group_algebra_even(observed_inner_mode)
    af = standard_model_algebra_signature()
    q8 = q8_real_group_algebra()
    witness = explicit_schur_witness()

    cyclic_can_match_af = bool(c14.get("H_blocks", 0) and c14.get("M3C_blocks", 0))
    dihedral_can_match_af = bool(d14.get("H_blocks", 0) and d14.get("M3C_blocks", 0))

    table = local_step4_table()
    for row in table:
        row["relative_defect"] = list(relative_defect(row))
    best_defect = min(tuple(row["relative_defect"]) for row in table)
    relative_winners = [row["algebra"] for row in table if tuple(row["relative_defect"]) == best_defect]

    # DSI-only no-go: all three plateau algebras admit precisely the same D-space.
    plateau = [row for row in table if row["dirac_dim"] == 16]
    dsi_only_unique = len({row["dirac_dim"] for row in plateau}) == len(plateau)
    # The expression above must be false: all values are 16.
    assert not dsi_only_unique

    schur = schur_lift_signature(1, 1, 3)

    receipt = {
        "status_grammar": {
            "image_mode_count": "COMPUTED_FROM_SUPPLIED_IMAGE",
            "spiral_model": "IDEALIZATION_NOT_ARCHAEOLOGICAL_PROOF",
            "group_algebra_no_go": "EXACT",
            "dsi_only_step4_no_go": "EXACT_CONDITIONAL_ON_REPORTED_DIRAC_SPACE_EQUALITY",
```

```
            "schur_lift": "EXACT_CONDITIONAL_THEOREM",
            "relative_step4": "CLOSED_ON_FIXED_CANDIDATE_CHAIN",
            "global_step4": "OPEN",
        },
        "image": {
            "path": str(IMAGE),
            "size_pixels": list(features["rgb"].shape[:2][::-1]),
            "center_xy_pixels": list(CENTER),
            "inner_annulus_pixels": list(INNER_ANNULUS),
            "outer_annulus_pixels": list(OUTER_ANNULUS),
            "inner_ranked_modes": inner_modes,
            "outer_ranked_modes": outer_modes,
            "strongest_inner_mode": observed_inner_mode,
            "strongest_outer_mode": observed_outer_mode,
            "robustness": {
                "inner_redness": inner_robust_red,
                "inner_edge": inner_robust_edge,
                "outer_redness": outer_robust_red,
                "outer_edge": outer_robust_edge,
            },
            "interpretation": "Mode 14 is the strongest nonzero inner-annulus harmonic; mode 8 is strongest in the outer field.
     Handwork, damage and photographic distortion prevent an exact symmetry claim from the image alone.",
        },
        "algebraic_no_go": {
            "R_Cn": c14,
            "R_Dn": d14,
            "A_F": af,
            "cyclic_ring_algebra_can_equal_A_F": cyclic_can_match_af,
            "dihedral_ring_algebra_can_equal_A_F": dihedral_can_match_af,
            "verdict": "The observed cyclic/dihedral ring symmetry has no quaternionic block and no 3x3 complex block; it cannot
     directly produce A_F.",
        },
        "quaternionic_branch": {
            "R_Q8": q8,
            "comparison": "R[D4] has R^4 + M2(R), whereas R[Q8] has R^4 + H. A quaternionic/spinorial lift can therefore supply the H
      block, but is not forced by the planar image."
        },
        "schur_lift": schur,
        "explicit_schur_witness": witness,
        "step4_local_chain": table,
        "relative_selector_winners": relative_winners,
        "dsi_only_unique_on_plateau": dsi_only_unique,
        "final_verdict": {
            "mosaic_alone_closes_standard_model": False,
            "decorated_schur_lift_reconstructs_A_F": True,
            "global_step4_closed": False,
        },
    }

    OUT_JSON.write_text(json.dumps(receipt, indent=2), encoding="utf-8")

    lines = [
        "FABLE v0.2 - mosaic-to-Standard-Model closure audit",
        "",
        f"Image centre: {CENTER}",
        f"Strongest inner angular mode: m={observed_inner_mode}",
        f"Strongest outer angular mode: m={observed_outer_mode}",
        f"Inner m=14 robustness (edge): {inner_robust_edge['target_is_top_count']}/{inner_robust_edge['trials']}",
        f"Outer m=8 robustness (edge): {outer_robust_edge['target_is_top_count']}/{outer_robust_edge['trials']}",
        "",
        f"R[C_{observed_inner_mode}] decomposition: R^{c14['R_blocks']} + C^{c14['C_blocks']}",
        f"R[D_{observed_inner_mode}] decomposition: R^{d14['R_blocks']} + M2(R)^{d14['M2R_blocks']}",
        "A_F target: C + H + M3(C)",
        "Direct group-algebra closure: FAIL",
        "",
        "DSI-only selector on the reported 16-dimensional Dirac plateau: FAIL",
        f"Relative selector winners: {', '.join(relative_winners)}",
        "Schur lift with types/multiplicities (C,1), (H,1), (C,3): CONDITIONAL PASS",
        f"Explicit C14 x Q8 commutant witness: {witness['verdict']} (dimension {witness['computed_commutant_dimension']})",
        "Global Step 4: OPEN",
    ]
    OUT_TXT.write_text("\n".join(lines) + "\n", encoding="utf-8")

    print(OUT_TXT.read_text(encoding="utf-8"))


if __name__ == "__main__":
    main()
```

F Code block F: original Light Matrix receipt

```
import numpy as np
from numpy.polynomial import polynomial as P
PHI=(1+5**.5)/2

M=np.array([[1,2,1,0],[1,1,0,0],[1,0,0,0],[0,0,0,1]],float)   # THEA's light matrix
Q=np.array([[1,1],[1,0]],float)                                 # Fibonacci step (k,l)->(k+l,k)

```

```
print("1. CLAIM: M_LIGHT = Sym^2(Q) (+) [1]  on state (k^2, kl, l^2, P)")
# symmetric square of Q in basis (k^2, kl, l^2)
a,b,c,d=Q[0,0],Q[0,1],Q[1,0],Q[1,1]
S2=np.array([[a*a,2*a*b,b*b],[a*c,a*d+b*c,b*d],[c*c,2*c*d,d*d]])
print("   Sym^2(Q) =\n", S2.astype(int))
print("   M_LIGHT[:3,:3] =\n", M[:3,:3].astype(int))
print("   identical:", np.array_equal(S2, M[:3,:3]), "| P block eigenvalue:", M[3,3])

print("\n2. eigenvalues")
print("   Q      :", np.sort(np.linalg.eigvals(Q)),  "   {phi, -1/phi} =", sorted([PHI,-1/PHI]))
print("   M_LIGHT:", np.sort(np.linalg.eigvals(M)),  "   {phi^2,1,-1,1/phi^2} =", sorted([PHI**2,1,-1,1/PHI**2]))
print("   det Q =", round(np.linalg.det(Q)), " -> area-preserving, ORIENTATION-REVERSING")

print("\n3. IS THE LADDER A SPIRAL OR A HYPERBOLA?")
print("   a similarity needs complex eigenvalues rho*e^{i*Delta}. Q's are REAL:")
w=np.linalg.eigvals(Q); print("   max |Im(eig Q)| =", abs(w.imag).max())
print("   -> Q is HYPERBOLIC (Anosov), not a similarity. Orbits lie on hyperbolas, not log spirals.")

print("\n4. Coxeter coordinates: w = k + l*exp(i*pi/3)  =>  |w|^2 = k^2+kl+l^2 = T ?")
om=np.exp(1j*np.pi/3)
bad=max(abs(abs(k+l*om)**2-(k*k+k*l+l*l)) for k in range(-6,7) for l in range(-6,7))
print("   max error over 169 lattice points:", f"{bad:.2e}", "-> EXACT")

print("\n5. golden branch (k,l)=(F_{n+1},F_n): T ladder and its ratio -> phi^2 ?")
k,l=1,0; Ts=[]
for n in range(12):
    Ts.append(k*k+k*l+l*l); k,l=k+l,k
print("   T =", Ts)
r=[Ts[i+1]/Ts[i] for i in range(4,11)]
print("   ratios ->", [round(x,6) for x in r], " phi^2 =", round(PHI**2,6))
print("   |ratio-phi^2| final =", f"{abs(r[-1]-PHI**2):.2e}")

print("\n6. T recurrence T_{n+3}=2T_{n+2}+2T_{n+1}-T_n on the golden branch?")
err=max(abs(Ts[n+3]-(2*Ts[n+2]+2*Ts[n+1]-Ts[n])) for n in range(len(Ts)-3))
print("   max residual over the ladder:", err, "-> EXACT" if err==0 else "-> FAILS")

print("\n7. Genesis topology closes on every ladder rung?")
ok=all((20*T)-(30*T)+(10*T+2)==2 for T in Ts)
print("   V-E+F = 20T-30T+(10T+2) = 2 for all T:", ok, "| P=12 is the eigenvalue-1 direction")

print("\n8. escape-time reading: T is multiplicative under Eisenstein mult?")
za,zb=(2+1j*0)+1*om, (1)+2*om
Ta=abs(za)**2; Tb=abs(zb)**2; Tab=abs(za*zb)**2
print(f"   T(w1)={Ta:.4f} T(w2)={Tb:.4f} T(w1*w2)={Tab:.4f} product={Ta*Tb:.4f} -> multiplicative:",
      abs(Tab-Ta*Tb)<1e-9)
print("   => squaring a seed SQUARES its triangulation number. escape count = GC refinement depth.")
```

References

[1] V. Savytskyy with Claude (Anthropic), *A Strange Idea, Followed Further: Step 6.3, Actually Done, and What That Showed*, May 2026, supplied manuscript.

[2] T. Krajewski, "Classification of Finite Spectral Triples," *J. Geom. Phys.* 28 (1998) 1–30.

[3] A. H. Chamseddine, A. Connes, and W. D. van Suijlekom, "Beyond the Spectral Standard Model: Emergence of Pati–Salam Unification," *JHEP* 11 (2013) 132.

[4] M. L. Lapidus and M. van Frankenhuijsen, *Fractal Geometry, Complex Dimensions and Zeta Functions*, Springer, 2nd ed.